

HOMEWORK 1 (124 points + 1) : CAUTION! CONTENTS ARE HOT

DUE: *September 22, 2026 @ 11:59 PM*

**24-HR LATE DUE DATE WITH A 15% PENALTY:
*September 23, 2026 @ 11:59 PM***

The [NCEI/WDS Global Significant Volcanic Eruptions Database](#) is a very comprehensive collection of +600 volcanic eruptions dating from 4360 BC to the present. Due to the nature of this assignment, we will be dealing with relatively newer volcanoes (in which some are still obviously still older than anyone on Earth currently). Each eruption in the database is classified as significant if it meets one or more criteria, such as causing fatalities, incurring **damage on property (+\$1 million)**, reaching a **Volcanic Explosivity Index (VEI) of 6 or higher**, generating a tsunami, or being linked to a significant earthquake. The VEI is a scale that measures the explosiveness of volcanic eruptions, providing insight into the magnitude and potential consequences of the eruptions. The database includes detailed information on the location, type of volcano, last known eruption, VEI, casualties, property damage, and much more.  volcano **We are going to dive straight into these volcanoes (well... their dataset), to swim our way into Pandas proficiency!**

You will find the [Pandas Documentation](#) helpful. There are also some helpful links to guide you along the way! Don't get burned

For this assignment, we are including a special bonus question to help everyone get familiar with the proper assignment submission routine. We strongly encourage you to complete Bonus Question 1, learn the process, and follow this practice for all future assignments.

For this assignment only, you will not lose points if you skip the bonus question. However, starting from Homework 2,

submitting both the Jupyter Notebook file and the PDF will be required in order to receive full credit.

DO NOT REMOVE ANY PART OF ANY OF THE QUESTIONS OR YOU LOSE CREDIT

No Hardcoding either

REMEMBER TO SHOW OUTPUT

Part 1: Maintenance (25 POINTS TOTAL)

First, we're going to familiarize ourselves with the process. As in most languages, Python looks best when its modules are imported first before any other code is written

```
In [1]: # Make sure these code blocks run properly and that you have properly installed
import pandas as pd
import requests
# import other libraries here

# Don't Remove this
pd.set_option('display.max_rows', None)
pd.set_option('display.max_columns', None)
```

As you may have noticed, there's another library aside from Pandas called "[requests](#)." **The requests library allows you to send HTTP requests to a server, retrieve the content, and process it at ease.** It's very beginner friendly for those attempting to get into webscraping (super important for collecting and creating datasets). We also recommend looking into [BeautifulSoup](#) (yeah, soup LOL), another wonderful library that can be paired with the requests library for webscraping.

As shown below, sometimes specific websites require specific headers in order to process a request to access the data.

To check if a request was processed successfully, use the [status_code](#) function to see if the process returned 200.

```
In [2]: # API URL and headers in case request gets denied.
api_url = "https://www.ngdc.noaa.gov/hazel/hazard-service/api/v1/volcanoes"

headers = {
    'accept': '*/*'
}
```

TASK 1.0: Cute Webscraping (5 points)

To make our cute webscraper we need to **create a GET request** using the relevant information given above.

This particular dataset NOAA returns data from the API as **json** when a user makes a request. The json data has a particular format, so we will extract our needed information only from the field called **items** to make a dataframe.

After properly scraping the data, name this dataframe *df*

Save this dataframe into a CSV file named 'volcanoes.csv'

You won't need to run this more than once

```
In [3]: # REMOVE THIS COMMENT AND ANSWER STARTING HERE, DONT DISPLAY THE DATASET
response = requests.get(api_url, headers = headers)
json_format = response.json()
list_volcanos = json_format["items"]
df = pd.DataFrame(list_volcanos)

df.to_csv('volcanoes.csv')
```

TASK 1.1: 1 Liner Thingz (3 points)

We need to get an idea of what this dataset is going to look. In order to do that, let's take a look at some of the most **basic things** our dataframe has.

Read the directions carefully and code your answer with only one line of code.

CAN'T USE LOOPS. DO NOT DISPLAY THE DATAFRAME, JUST YOUR CODE OUTPUT HERE.

1.1.1: In one line of code and **using only one single method call**, output the **summary statistics** (count, mean, std, min, max, quartiles) of all numerical features in the dataframe.

Hint: Look for a method that gives a descriptive overview of numerical columns.

```
In [4]: # REMOVE THIS COMMENT AND ANSWER STARTING HERE, DONT DISPLAY THE DATASET
df.describe()
```

Out [4]:

	id	year	month	day	tsunamiEventId	earthquake
count	200.000000	200.000000	177.000000	155.000000	37.000000	17.
mean	375.635000	1809.800000	6.293785	15.825806	1818.648649	3632
std	306.114755	248.034282	3.469611	9.185073	1742.563313	3015
min	1.000000	-141.000000	1.000000	1.000000	61.000000	421.
25%	78.500000	1774.500000	3.000000	8.000000	613.000000	1328.
50%	348.000000	1880.500000	7.000000	15.000000	1181.000000	2179.
75%	664.250000	1950.000000	9.000000	24.000000	1977.000000	6195.
max	994.000000	1996.000000	12.000000	31.000000	5863.000000	10519.

1.1.2: In one line of code and **using only one single method call**, print the **summary information** of the dataframe, including index dtype, column names, non-null counts, and memory usage.

```
In [5]: # REMOVE THIS COMMENT AND ANSWER STARTING HERE, DONT DISPLAY THE DATASET
df.info()
```

```

<class 'pandas.DataFrame'>
RangeIndex: 200 entries, 0 to 199
Data columns (total 43 columns):
#   Column                                     Non-Null Count  Dtype
---  -
0   id                                           200 non-null    int64
1   year                                         200 non-null    int64
2   month                                       177 non-null    float64
3   day                                          155 non-null    float64
4   tsunamiEventId                             37 non-null     float64
5   earthquakeEventId                           17 non-null     float64
6   volcanoLocationId                           200 non-null    int64
7   volcanoLocationNewNum                       200 non-null    int64
8   volcanoLocationNum                           200 non-null    str
9   name                                         200 non-null    str
10  location                                     200 non-null    str
11  country                                     200 non-null    str
12  latitude                                    200 non-null    float64
13  longitude                                   200 non-null    float64
14  elevation                                   200 non-null    int64
15  morphology                                   200 non-null    str
16  agent                                         128 non-null    str
17  deathsTotal                                 128 non-null    float64
18  deathsAmountOrderTotal                     180 non-null    float64
19  damageAmountOrderTotal                     59 non-null     float64
20  significant                                  200 non-null    bool
21  publish                                     200 non-null    bool
22  eruption                                    200 non-null    bool
23  status                                       200 non-null    str
24  timeErupt                                   200 non-null    str
25  vei                                          192 non-null    float64
26  deathsAmountOrder                           169 non-null    float64
27  damageAmountOrder                           51 non-null     float64
28  housesDestroyedAmountOrderTotal             52 non-null     float64
29  deaths                                       118 non-null    float64
30  injuries                                    30 non-null     float64
31  injuriesAmountOrder                         39 non-null     float64
32  injuriesTotal                               33 non-null     float64
33  injuriesAmountOrderTotal                   41 non-null     float64
34  housesDestroyedAmountOrder                 42 non-null     float64
35  housesDestroyed                             13 non-null     float64
36  housesDestroyedTotal                       16 non-null     float64
37  missingAmountOrder                         3 non-null      float64
38  missingAmountOrderTotal                    3 non-null      float64
39  missing                                      1 non-null      float64
40  missingTotal                               1 non-null      float64
41  damageMillionsDollars                      1 non-null      float64
42  damageMillionsDollarsTotal                 1 non-null      float64
dtypes: bool(3), float64(27), int64(5), str(8)
memory usage: 63.2 KB

```

We won't be using some of the data because there is a lot of missing data.

1.1.3: In one line of code, create a **new dataframe** called **new_df** that **discards** all the features of the **old** dataframe **except for the following**:

```
id, year, month, day, tsunamiEventId, earthquakeEventId,
volcanoLocationId, volcanoLocationNewNum, name, country, elevation,
morphology, deathsTotal, vei, deaths
```

Hint: Don't use any drop function

```
In [6]: new_df = df[["id", "year", "month", "day", "tsunamiEventId", "e
```

TASK 1.2: 1 Liner Shenaniganz (7 points)

We're going to tidy up the **new dataframe** a little more with some more advanced 1 liner code.

Read the directions carefully and code your answer with only one line of code.

For this section, keep the method of display that is already in the box. Write your code as indicated.

YOU CAN'T USE ONE LINE LOOPS OR ANY KIND OF LOOP.

1.2.1: *In one line of code, **drop any row** that contains **NaN** in **any one** of the columns indicating a measure of **time**.*

```
In [7]: # REMOVE THIS COMMENT AND ANSWER STARTING HERE, DONT DISPLAY THE DATASET
new_df = new_df.dropna(subset=["year", "day", "month"])
```

1.2.2: *In one line of code, **change** the **index column** of the dataframe so that it has **1-based indexing**.*

```
In [8]: # REMOVE THIS COMMENT AND ANSWER STARTING HERE, DONT DISPLAY THE DATASET
new_df.index = new_df.index + 1
```

The **deathsTotal** and **deaths** columns have approximations of the same data with alternating NaNs in each.

1.2.3: *In one line of code, make a **new column** called '**totalDeaths**' that takes the **max** of the values given between those **two** *columns.*

- If there is **NaN** in **one column** and a **numerical** value in the **other**, it will **take the numerical value**.
- **Only** if there are **NaNs** in **both columns**, the **new column** will have **NaN**.

```
In [9]: # REMOVE THIS COMMENT AND ANSWER STARTING HERE, DONT DISPLAY THE DATASET
new_df["totalDeaths"] = new_df[["deathsTotal", "deaths"]].max(axis = 1)
```

TASK 1.3: Tailoring Time (10 Points)

It's pretty obvious that the year, month, and day look pretty weird in the dataset. We're going to have to do some hardcore cleaning on the [time](#).

We need to have only ONE column called "date" that contains the full date (YYYY-MM-DD), not separated into three columns.

Make sure there are no floating point values in the date and sort the data from most recent to least.

Remove the old columns and place the new column next to the 'id' column.

YOU CAN USE MULTIPLE LINES OF CODE BUT CAN'T USE LOOPS.

Note: It is alright to have only a **maximum of 12 NaTs** for some dates that often go further back than the 1600s because the datetime module in Pandas has a limit (unless otherwise guided).

```
In [10]: # REMOVE THIS COMMENT AND ANSWER STARTING HERE

new_df['date'] = pd.to_datetime(new_df[['year', 'month', 'day']])

# Drop original columns and sort.
new_df = new_df.drop(columns = ['year', 'month', 'day']) # drop original
new_df = new_df.sort_values(by='date', ascending = False)

# Place new column next to 'id' column.

new_df = new_df[["id", "date", "tsunamiEventId", "earthquakeEventId", "volca

# KEEP THIS. It will display the whole dataframe.
new_df
```

Out[10]:

	id	date	tsunamiEventId	earthquakeEventId	volcanoLocationId	volcanoLocat
185	844	7660-07-20	61.0	7341.0	80208	
157	693	1996-08-10	NaN	NaN	70202	
60	112	1994-11-22	NaN	NaN	60325	
196	933	1991-06-03	NaN	NaN	80210	
75	144	1990-02-10	NaN	NaN	60328	
123	486	1988-05-09	NaN	NaN	60509	
31	31	1987-04-17	NaN	NaN	10106	
59	111	1986-12-31	NaN	NaN	60325	
13	13	1986-07-24	NaN	NaN	10104	
88	203	1985-05-10	NaN	NaN	60330	
172	739	1984-09-09	NaN	NaN	70303	
40	40	1984-08-15	5807.0	NaN	20403	
114	447	1983-08-17	4702.0	NaN	60425	
124	488	1983-07-23	NaN	NaN	60601	
108	398	1981-09-05	NaN	NaN	60415	
91	206	1981-03-29	NaN	NaN	60330	
30	30	1979-09-12	NaN	NaN	10106	
20	20	1979-03-08	NaN	NaN	50103	

	id	date	tsunamiEventId	earthquakeEventId	volcanoLocationId	volcanoLocat
46	49	1979-02-20	NaN	NaN	60320	
89	204	1978-09-19	NaN	NaN	60330	
159	698	1978-07-29	NaN	NaN	70301	
7	7	1977-01-10	NaN	NaN	20303	
131	562	1976-09-15	NaN	NaN	60702	
90	205	1976-08-31	NaN	NaN	60330	
130	561	1974-02-11	NaN	NaN	60702	
21	21	1970-03-02	NaN	NaN	10101	
180	777	1969-03-21	2872.0	NaN	70402	
104	389	1969-01-28	NaN	NaN	60411	
58	105	1969-01-07	NaN	NaN	60325	
171	736	1968-04-21	NaN	NaN	70303	
87	202	1967-08-31	NaN	NaN	60330	
138	589	1966-08-12	NaN	NaN	60704	
74	141	1966-04-26	NaN	NaN	60328	
179	762	1965-09-28	1977.0	NaN	70307	
106	395	1964-01-01	NaN	NaN	60415	
100	345	1963-09-05	NaN	NaN	60401	

	id	date	tsunamiEventId	earthquakeEventId	volcanoLocationId	volcanoLocat
102	353	1963-05-16	NaN	4292.0	60402	
86	201	1963-05-05	NaN	NaN	60330	
101	351	1963-03-18	1942.0	NaN	60402	
148	657	1962-12-31	NaN	NaN	60806	
57	104	1961-05-08	NaN	NaN	60325	
126	525	1958-07-12	NaN	NaN	60611	
191	878	1955-10-13	NaN	NaN	80208	
38	38	1954-08-03	NaN	NaN	50101	
56	100	1954-01-18	NaN	NaN	60325	
73	140	1951-08-31	NaN	NaN	60328	
156	680	1950-09-15	NaN	NaN	70108	
85	200	1950-08-28	NaN	NaN	60330	
112	441	1948-04-07	NaN	NaN	60425	
170	735	1947-01-07	NaN	NaN	70303	
84	199	1946-10-29	NaN	NaN	60330	
45	45	1944-12-04	NaN	NaN	60320	
39	39	1944-03-27	NaN	NaN	10102	
82	196	1941-09-21	NaN	NaN	60330	

	id	date	tsunamiEventId	earthquakeEventId	volcanoLocationId	volcanoLocat
129	552	1940-06-20	NaN	NaN	60702	
169	733	1938-06-05	NaN	NaN	70303	
158	697	1933-12-25	1666.0	NaN	70301	
183	823	1933-12-24	NaN	NaN	80205	
55	94	1930-12-18	NaN	NaN	60325	
26	26	1930-09-11	4218.0	NaN	10104	
28	28	1928-11-08	NaN	NaN	10106	
105	394	1928-08-04	1607.0	NaN	60415	
42	42	1926-01-23	NaN	NaN	10204	
54	92	1920-07-25	NaN	NaN	60325	
25	25	1919-05-22	1476.0	NaN	10104	
72	138	1919-05-19	NaN	NaN	60328	
18	18	1917-04-01	NaN	NaN	40105	
32	32	1914-09-10	NaN	NaN	40104	
189	867	1914-01-12	1412.0	2994.0	80208	
81	195	1913-06-23	NaN	NaN	60330	
137	585	1913-03-14	1407.0	2973.0	60704	
34	34	1912-12-03	NaN	NaN	20302	
80	193	1911-11-08	NaN	NaN	60330	

	id	date	tsunamiEventId	earthquakeEventId	volcanoLocationId	volcanoLocat
178	761	1911-01-30	1392.0	8391.0	70307	
79	191	1909-08-29	NaN	NaN	60330	
110	411	1907-09-28	NaN	NaN	60418	
11	11	1907-08-04	NaN	NaN	20112	
12	12	1905-03-10	NaN	NaN	10102	
15	15	1904-02-25	NaN	NaN	30301	
27	27	1903-08-30	NaN	NaN	40105	
71	137	1901-05-22	NaN	NaN	60328	
167	730	1897-05-23	NaN	NaN	70303	
35	35	1895-06-17	NaN	NaN	50103	
78	176	1895-05-22	NaN	NaN	60330	
136	584	1892-06-07	1197.0	NaN	60704	
153	669	1890-06-29	NaN	NaN	60807	
132	569	1889-09-06	1181.0	6309.0	60703	
4	4	1888-03-13	1175.0	NaN	50107	
166	724	1887-03-09	NaN	NaN	70303	
33	33	1886-08-31	NaN	NaN	40311	

	id	date	tsunamiEventId	earthquakeEventId	volcanoLocationId	volcanoLocat
77	167	1885-04-18	NaN	NaN	60330	
16	16	1878-02-04	1095.0	6195.0	50214	
70	136	1875-01-29	NaN	NaN	60328	
177	756	1874-07-19	NaN	NaN	70307	
154	677	1873-01-16	NaN	NaN	70106	
52	82	1872-11-03	NaN	NaN	60325	
29	29	1872-04-24	NaN	NaN	10102	
51	81	1872-04-15	NaN	NaN	60325	
165	716	1871-12-08	NaN	NaN	70303	
147	643	1871-08-07	NaN	NaN	60806	
155	679	1871-04-30	1020.0	2179.0	70108	
128	533	1871-03-03	1022.0	NaN	60701	
109	410	1869-07-07	NaN	NaN	60418	
69	135	1864-01-04	NaN	NaN	60328	
152	666	1861-12-29	NaN	NaN	60807	
135	581	1856-03-02	859.0	1995.0	60704	
181	787	1853-10-29	809.0	NaN	80103	
163	712	1853-07-13	NaN	NaN	70303	

	id	date	tsunamiEventId	earthquakeEventId	volcanoLocationId	volcanoLocat
37	37	1853-06-24	NaN	NaN	40311	
68	133	1848-05-16	NaN	NaN	60328	
125	489	1845-02-08	741.0	1865.0	60603	
44	44	1843-11-25	NaN	NaN	10106	
146	629	1840-02-02	718.0	NaN	60806	
145	627	1838-02-26	NaN	NaN	60806	
50	71	1832-12-25	NaN	NaN	60325	
6	6	1832-11-01	NaN	NaN	10106	
67	132	1826-10-13	NaN	NaN	60328	
49	69	1822-12-27	NaN	NaN	60325	
122	482	1820-06-11	NaN	NaN	60509	
99	333	1817-01-24	NaN	NaN	60335	
103	367	1815-04-10	613.0	NaN	60404	
162	707	1814-02-01	NaN	NaN	70303	
134	580	1812-08-06	NaN	NaN	60704	
24	24	1805-08-11	NaN	NaN	10102	
161	706	1800-10-30	NaN	NaN	70303	
195	932	1792-05-21	568.0	1515.0	80210	

	id	date	tsunamiEventId	earthquakeEventId	volcanoLocationId	volcanoLocat
188	859	1781-04-11	529.0	NaN	80208	
187	858	1779-11-08	NaN	NaN	80208	
9	9	1779-08-08	NaN	NaN	10102	
143	618	1773-02-02	NaN	NaN	60806	
142	617	1772-05-09	504.0	NaN	60806	
160	704	1766-07-20	NaN	NaN	70303	
176	754	1754-11-28	2853.0	1328.0	70307	
175	753	1749-08-11	431.0	1304.0	70307	
17	17	1737-05-20	NaN	NaN	10102	
194	916	1717-02-07	NaN	NaN	80209	
193	915	1716-11-09	NaN	NaN	80209	
174	752	1716-09-24	387.0	1205.0	70307	
66	128	1716-07-20	NaN	NaN	60328	
133	578	1711-12-11	NaN	NaN	60704	
121	477	1694-11-20	NaN	NaN	60509	
118	466	1692-06-04	NaN	NaN	60507	
23	23	1682-08-12	NaN	NaN	10102	
140	599	1673-05-20	312.0	NaN	60804	

	id	date	tsunamiEventId	earthquakeEventId	volcanoLocationId	volcanoLocat
48	64	1672-08-04	NaN	NaN	60325	
116	457	1659-11-11	285.0	NaN	60505	
150	664	1646-07-19	5754.0	10519.0	60807	
43	43	1631-02-14	NaN	920.0	201141	
141	607	1608-07-18	222.0	NaN	60806	
192	904	1566-10-31	NaN	NaN	80209	
36	36	1538-09-29	NaN	NaN	10101	
14	14	1536-03-26	NaN	NaN	10106	
186	846	1471-11-03	NaN	NaN	80208	
2	2	1329-07-15	NaN	NaN	10106	
1	1	1169-02-04	2852.0	421.0	10106	

Part 2: Volcanic Matryoshkas (30 POINTS TOTAL)

Now, that most of the data has been tidied up. We will organize the data into more sizable pieces of information in order to extract useful information.

2.1.1: (10 points here)

Use the **groupby function in Pandas** to create separate dataframes for each unique country.

- **Each table must only have the columns:** 'date' 'country', 'name', and 'vei' (Having the index is ok. Up to you.)
- **Sort** the dataframe of **each country** by **highest to lowest 'vei'**
- Use the **display** function to show **each sorted table**

You MUST use the groupby function here and display your results.

```
In [11]: from IPython.display import display
# REMOVE THIS COMMENT AND ANSWER STARTING HERE

smaller_df = new_df[['date', 'country', 'name', 'vei']]
grouped = smaller_df.groupby('country')

for country_name, country_mini_df in grouped:
    sorted_group = country_mini_df.sort_values(by = 'vei', ascending = False)
    display(sorted_group)
```

	date	country	name	vei
40	1984-08-15	Cameroon	Oku Volcanic Field	NaN

	date	country	name	vei
15	1904-02-25	Comoros	Karthala	2.0

	date	country	name	vei
34	1912-12-03	Congo, DRC	Nyamulagira	3.0
7	1977-01-10	Congo, DRC	Nyiragongo	1.0

	date	country	name	vei
11	1907-08-04	Ethiopia	Alayta	2.0
43	1631-02-14	Ethiopia	Dama Ali	NaN

	date	country	name	vei
42	1926-01-23	Greece	Santorini	2.0

	date	country	name	vei
103	1815-04-10	Indonesia	Tambora	7.0
140	1673-05-20	Indonesia	Gamkonora	5.0
101	1963-03-18	Indonesia	Agung	5.0
51	1872-04-15	Indonesia	Merapi	4.0
138	1966-08-12	Indonesia	Awu	4.0
152	1861-12-29	Indonesia	Kie Besi	4.0
52	1872-11-03	Indonesia	Merapi	4.0
73	1951-08-31	Indonesia	Kelud	4.0
67	1826-10-13	Indonesia	Kelud	4.0
72	1919-05-19	Indonesia	Kelud	4.0
75	1990-02-10	Indonesia	Kelud	4.0
124	1983-07-23	Indonesia	Colo	4.0
150	1646-07-19	Indonesia	Kie Besi	4.0
74	1966-04-26	Indonesia	Kelud	4.0
71	1901-05-22	Indonesia	Kelud	3.0
80	1911-11-08	Indonesia	Semeru	3.0
136	1892-06-07	Indonesia	Awu	3.0
105	1928-08-04	Indonesia	Paluweh	3.0
68	1848-05-16	Indonesia	Kelud	3.0
135	1856-03-02	Indonesia	Awu	3.0
146	1840-02-02	Indonesia	Gamalama	3.0
49	1822-12-27	Indonesia	Merapi	3.0
134	1812-08-06	Indonesia	Awu	3.0
142	1772-05-09	Indonesia	Gamalama	3.0
133	1711-12-11	Indonesia	Awu	3.0
48	1672-08-04	Indonesia	Merapi	3.0
55	1930-12-18	Indonesia	Merapi	3.0
60	1994-11-22	Indonesia	Merapi	3.0
141	1608-07-18	Indonesia	Gamalama	3.0
88	1985-05-10	Indonesia	Semeru	3.0
87	1967-08-31	Indonesia	Semeru	3.0
91	1981-03-29	Indonesia	Semeru	3.0

	date	country	name	vei
104	1969-01-28	Indonesia	Iya	3.0
89	1978-09-19	Indonesia	Semeru	3.0
57	1961-05-08	Indonesia	Merapi	3.0
102	1963-05-16	Indonesia	Agung	3.0
130	1974-02-11	Indonesia	Karangetang	3.0
145	1838-02-26	Indonesia	Gamalama	2.0
69	1864-01-04	Indonesia	Kelud	2.0
58	1969-01-07	Indonesia	Merapi	2.0
125	1845-02-08	Indonesia	Soputan	2.0
131	1976-09-15	Indonesia	Karangetang	2.0
50	1832-12-25	Indonesia	Merapi	2.0
90	1976-08-31	Indonesia	Semeru	2.0
128	1871-03-03	Indonesia	Ruang	2.0
99	1817-01-24	Indonesia	Ijen	2.0
143	1773-02-02	Indonesia	Gamalama	2.0
66	1716-07-20	Indonesia	Kelud	2.0
108	1981-09-05	Indonesia	Paluweh	2.0
59	1986-12-31	Indonesia	Merapi	2.0
109	1869-07-07	Indonesia	Lewotobi	2.0
147	1871-08-07	Indonesia	Gamalama	2.0
112	1948-04-07	Indonesia	Iliwerung	2.0
106	1964-01-01	Indonesia	Paluweh	2.0
84	1946-10-29	Indonesia	Semeru	2.0
45	1944-12-04	Indonesia	Dieng Volcanic Complex	2.0
82	1941-09-21	Indonesia	Semeru	2.0
129	1940-06-20	Indonesia	Karangetang	2.0
56	1954-01-18	Indonesia	Merapi	2.0
126	1958-07-12	Indonesia	Mahawu	2.0
54	1920-07-25	Indonesia	Merapi	2.0
81	1913-06-23	Indonesia	Semeru	2.0
137	1913-03-14	Indonesia	Awu	2.0
148	1962-12-31	Indonesia	Gamalama	2.0

	date	country	name	vei
79	1909-08-29	Indonesia	Semeru	2.0
110	1907-09-28	Indonesia	Lewotobi	2.0
86	1963-05-05	Indonesia	Semeru	2.0
78	1895-05-22	Indonesia	Semeru	2.0
153	1890-06-29	Indonesia	Kie Besi	2.0
132	1889-09-06	Indonesia	Banua Wuhu	2.0
77	1885-04-18	Indonesia	Semeru	2.0
100	1963-09-05	Indonesia	Batur	2.0
46	1979-02-20	Indonesia	Dieng Volcanic Complex	1.0
114	1983-08-17	Indonesia	Iliwerung	1.0
85	1950-08-28	Indonesia	Semeru	1.0
70	1875-01-29	Indonesia	Kelud	NaN

	date	country	name	vei
2	1329-07-15	Italy	Etna	3.0
9	1779-08-08	Italy	Vesuvius	3.0
30	1979-09-12	Italy	Etna	3.0
39	1944-03-27	Italy	Vesuvius	3.0
26	1930-09-11	Italy	Stromboli	3.0
14	1536-03-26	Italy	Etna	3.0
25	1919-05-22	Italy	Stromboli	3.0
36	1538-09-29	Italy	Campi Flegrei	3.0
29	1872-04-24	Italy	Vesuvius	3.0
23	1682-08-12	Italy	Vesuvius	3.0
17	1737-05-20	Italy	Vesuvius	3.0
31	1987-04-17	Italy	Etna	2.0
24	1805-08-11	Italy	Vesuvius	2.0
6	1832-11-01	Italy	Etna	2.0
13	1986-07-24	Italy	Stromboli	2.0
12	1905-03-10	Italy	Vesuvius	2.0
44	1843-11-25	Italy	Etna	2.0
28	1928-11-08	Italy	Etna	1.0
21	1970-03-02	Italy	Campi Flegrei	NaN
1	1169-02-04	Italy	Etna	NaN

	date	country	name	vei
186	1471-11-03	Japan	Aira	5.0
183	1933-12-24	Japan	Kuchinoerabujima	4.0
189	1914-01-12	Japan	Aira	4.0
188	1781-04-11	Japan	Aira	4.0
187	1779-11-08	Japan	Aira	4.0
185	7660-07-20	Japan	Aira	3.0
191	1955-10-13	Japan	Aira	3.0
194	1717-02-07	Japan	Kirishimayama	3.0
193	1716-11-09	Japan	Kirishimayama	3.0
192	1566-10-31	Japan	Kirishimayama	3.0
195	1792-05-21	Japan	Unzendake	2.0
196	1991-06-03	Japan	Unzendake	1.0

	date	country	name	vei
18	1917-04-01	New Zealand	Okataina	1.0
27	1903-08-30	New Zealand	Okataina	1.0
32	1914-09-10	New Zealand	Whakaari/White Island	NaN

	date	country	name	vei
118	1692-06-04	Pacific Ocean	Serua	4.0
116	1659-11-11	Pacific Ocean	Teon	4.0
123	1988-05-09	Pacific Ocean	Banda Api	3.0
121	1694-11-20	Pacific Ocean	Banda Api	3.0
122	1820-06-11	Pacific Ocean	Banda Api	2.0

	date	country	name	vei
16	1878-02-04	Papua New Guinea	Rabaul	3.0
20	1979-03-08	Papua New Guinea	Karkar	2.0
38	1954-08-03	Papua New Guinea	Bam	2.0
35	1895-06-17	Papua New Guinea	Karkar	2.0
4	1888-03-13	Papua New Guinea	Ritter Island	2.0

	date	country	name	vei
174	1716-09-24	Philippines	Taal	4.0
176	1754-11-28	Philippines	Taal	4.0
179	1965-09-28	Philippines	Taal	4.0
162	1814-02-01	Philippines	Mayon	4.0
167	1897-05-23	Philippines	Mayon	3.0
175	1749-08-11	Philippines	Taal	3.0
160	1766-07-20	Philippines	Mayon	3.0
163	1853-07-13	Philippines	Mayon	3.0
165	1871-12-08	Philippines	Mayon	3.0
172	1984-09-09	Philippines	Mayon	3.0
166	1887-03-09	Philippines	Mayon	3.0
178	1911-01-30	Philippines	Taal	3.0
156	1950-09-15	Philippines	Camiguin	3.0
171	1968-04-21	Philippines	Mayon	3.0
158	1933-12-25	Philippines	Bulusan	2.0
177	1874-07-19	Philippines	Taal	2.0
154	1873-01-16	Philippines	Ragang	2.0
169	1938-06-05	Philippines	Mayon	2.0
155	1871-04-30	Philippines	Camiguin	2.0
170	1947-01-07	Philippines	Mayon	2.0
161	1800-10-30	Philippines	Mayon	2.0
180	1969-03-21	Philippines	Didicas	2.0
159	1978-07-29	Philippines	Bulusan	2.0
157	1996-08-10	Philippines	Kanlaon	2.0

	date	country	name	vei
181	1853-10-29	Taiwan	Unnamed	2.0

	date	country	name	vei
33	1886-08-31	Tonga	Niuafo'ou	4.0
37	1853-06-24	Tonga	Niuafo'ou	0.0

2.1.2: (5 points here)

Using **groupby** again, print out the **maximum 'vei'** for **each unique country**.

You MUST use the groupby function here and print your results.

- **Print** out your results in a format like the following: "Country: {country_name}, Highest VEI: {vei}"

```
In [12]: # REMOVE THIS COMMENT AND ANSWER STARTING HERE
max_vei_country = grouped['vei'].max()

#items() gives us the index and values of a series
for country_name, vei in max_vei_country.items():
    print(f"Country: {country_name}, Highest VEI: {vei}")
```

```
Country: Cameroon, Highest VEI: nan
Country: Comoros, Highest VEI: 2.0
Country: Congo, DRC, Highest VEI: 3.0
Country: Ethiopia, Highest VEI: 2.0
Country: Greece, Highest VEI: 2.0
Country: Indonesia, Highest VEI: 7.0
Country: Italy, Highest VEI: 3.0
Country: Japan, Highest VEI: 5.0
Country: New Zealand, Highest VEI: 1.0
Country: Pacific Ocean, Highest VEI: 4.0
Country: Papua New Guinea, Highest VEI: 3.0
Country: Philippines, Highest VEI: 4.0
Country: Taiwan, Highest VEI: 2.0
Country: Tonga, Highest VEI: 4.0
```

Finally, we have ALMOST REACHED THE END!! Since there is still quite a bit of missing data, we want to make use of what is still available.

A very powerful tool in Python's magnificent collection of libraries is its beautiful graphing tools.

Check out libraries such as [Seaborn](#) or [Matplotlib](#) to create meaningful visualizations!

Your final task in this section requires the use of these libraries

2.1.3: (15 points here)

- Based on the **unique names of volcanoes**, filter names that have more than 4 **datapoints under their name**.
- Each datapoint in the dataframe refers to a recorded instance of a volcanic eruption.
- Make **separate line graphs for each volcano** and **plot their VEIs over time**.

Make sure to properly label all parts of the graph appropriately to receive credit 🙄
(like title, axes, legend, etc...)

```
In [16]: # REMOVE THIS COMMENT AND ANSWER STARTING HERE
import matplotlib.pyplot as plt

frequency_count_series = new_df.value_counts('name') #getting the volcanos r
```

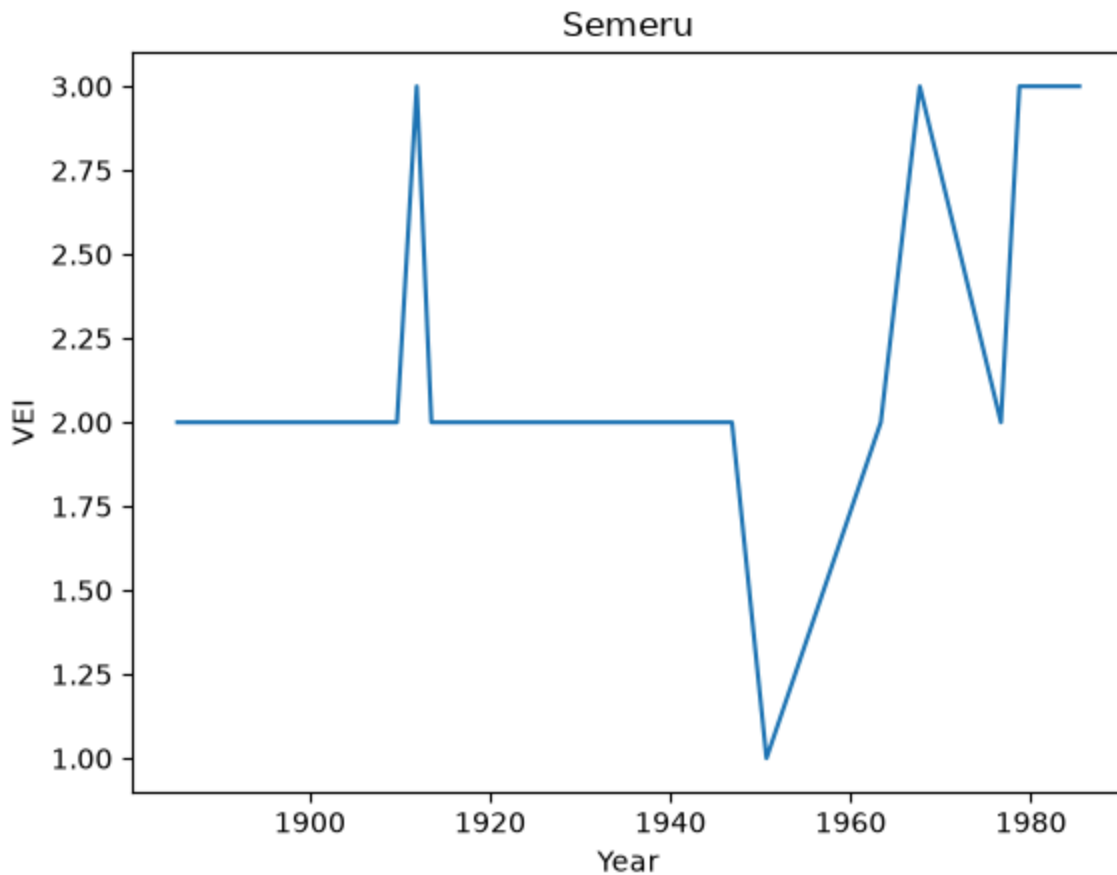
```

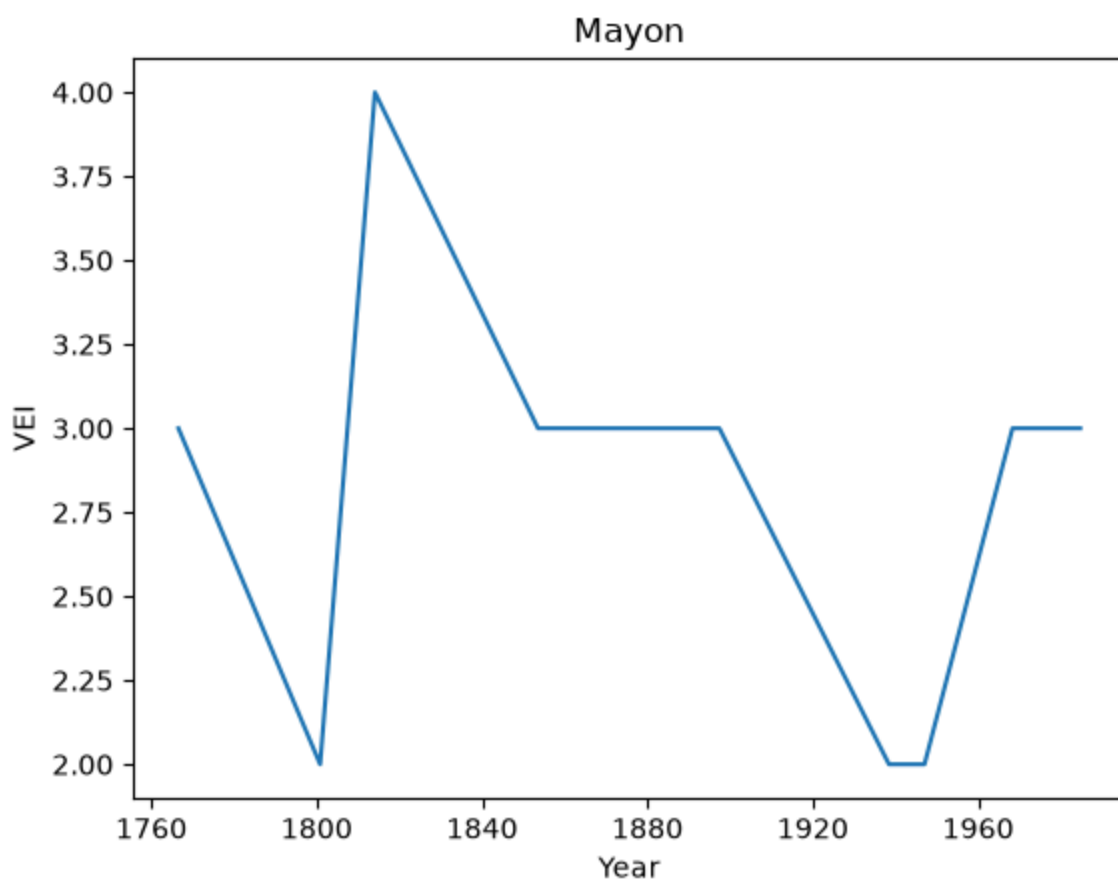
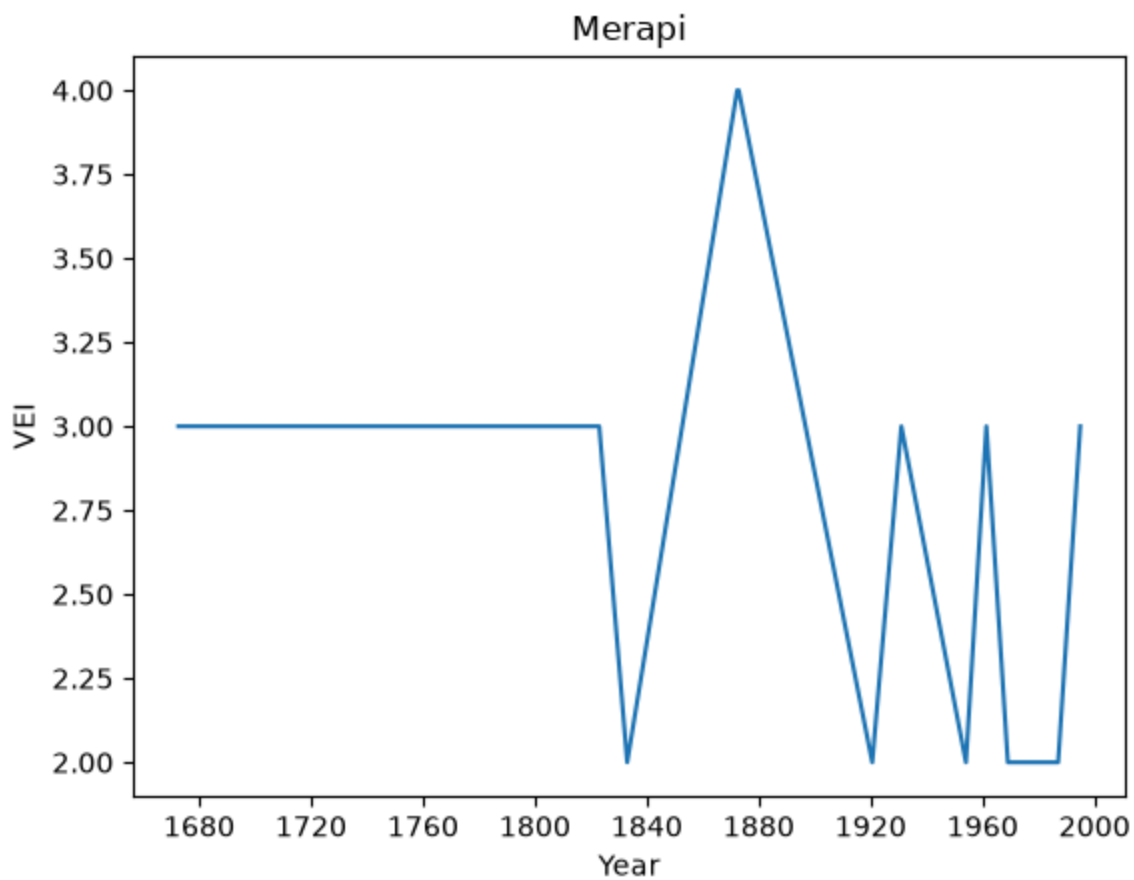
frequency_count_series = frequency_count_series[frequency_count_series > 4]
qualified_names = frequency_count_series.index #just the names of the qualified
final_filtered_with_columns = new_df[new_df['name'].isin(qualified_names)] #

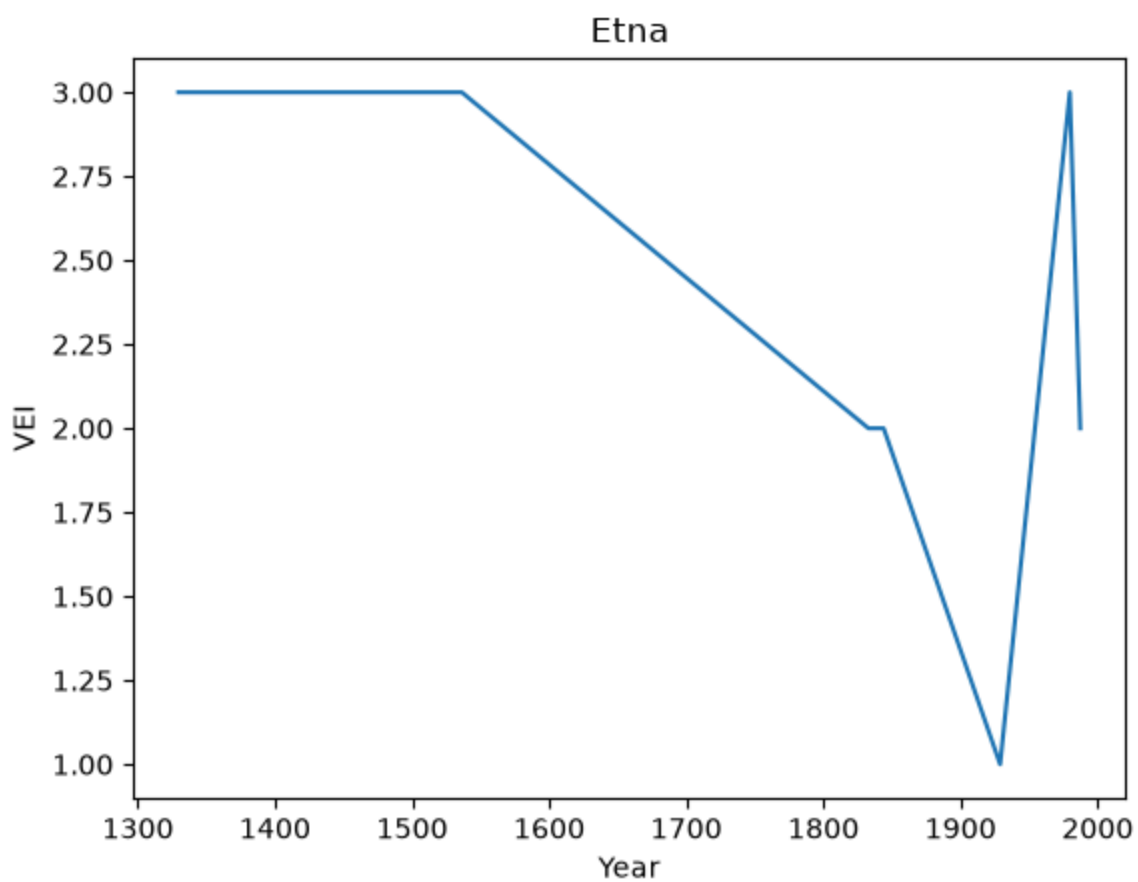
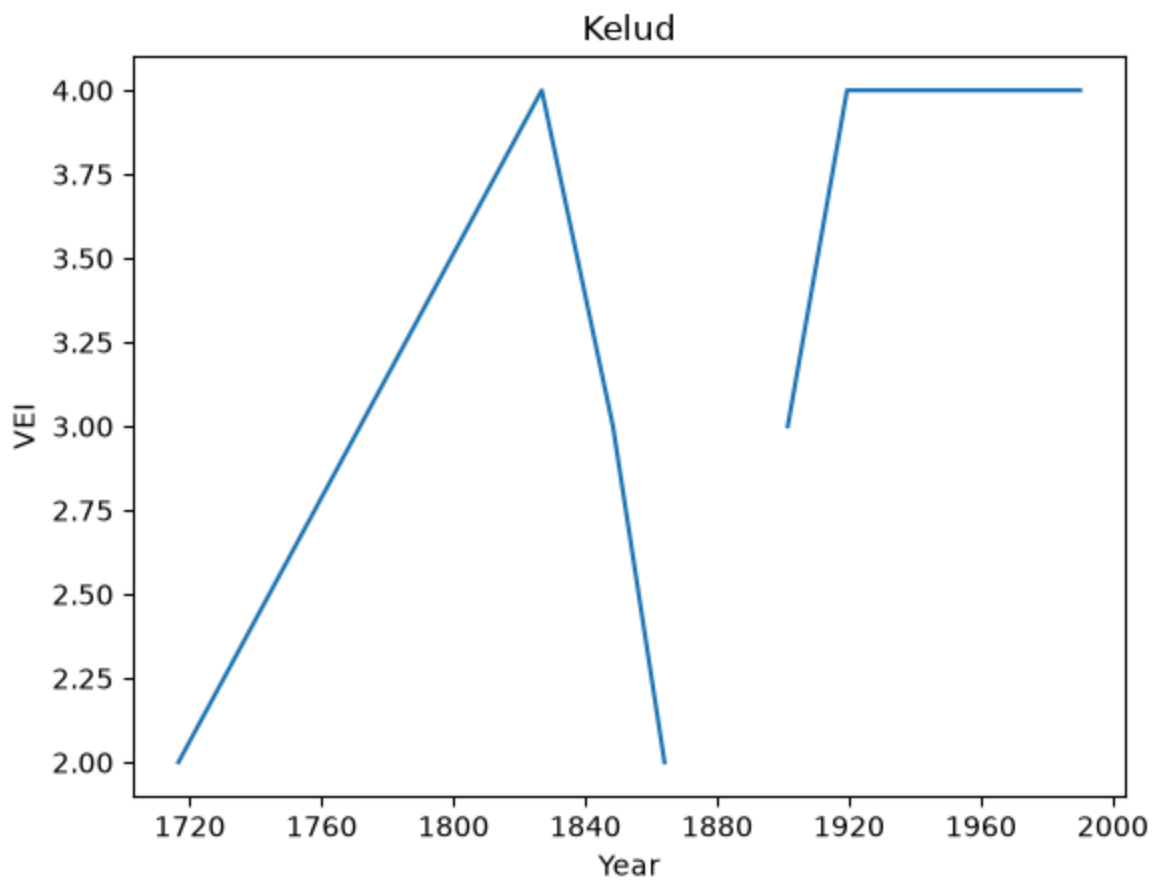
#writing a for loop and go through each one and plot them
for volcano_name in qualified_names:
    grouped_volcano_df = final_filtered_with_columns[final_filtered_with_col
    #now we need to graph them
    plt.figure() # creating the graph stand
    plt.plot(grouped_volcano_df['date'], grouped_volcano_df['vei'])
    plt.title(volcano_name)
    plt.xlabel('Year')
    plt.ylabel('VEI')
    plt.show()

# not adding the graphs here, but the graphs should have the volcano name as
# the x axis should be the years and the y axis should be the VEI
# A graph that doesn't have these things will only get partial points

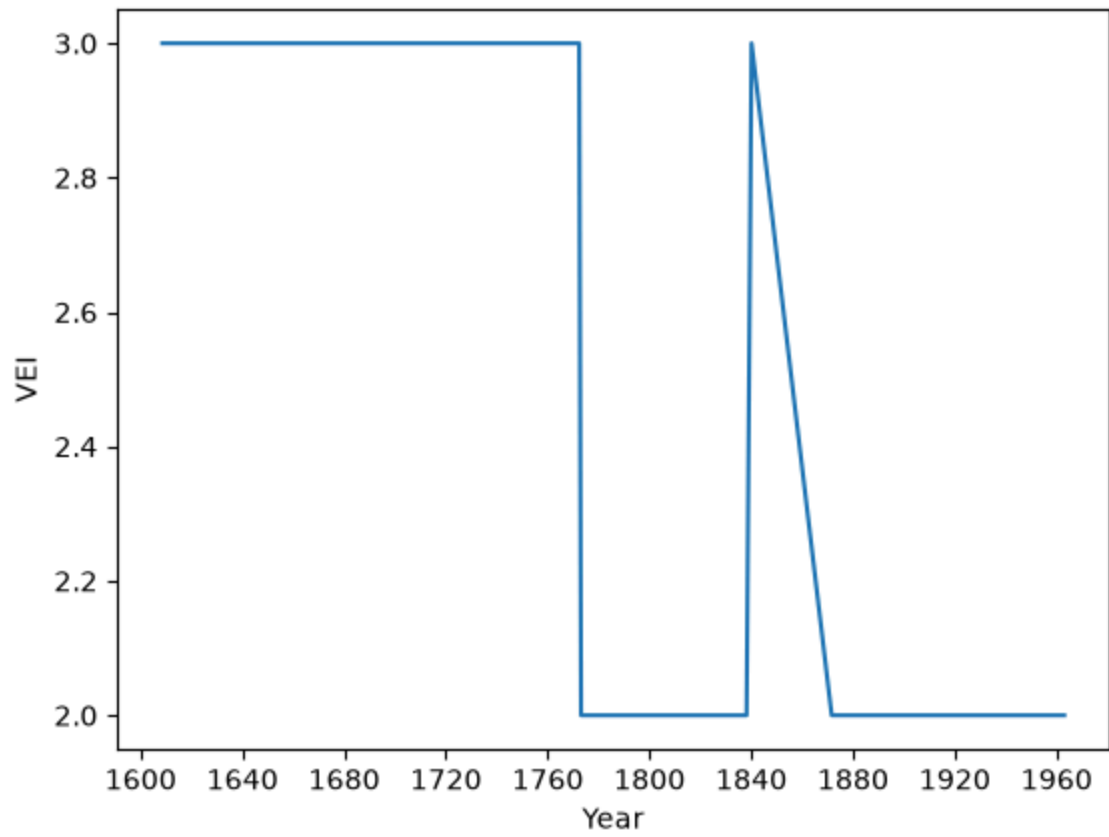
```



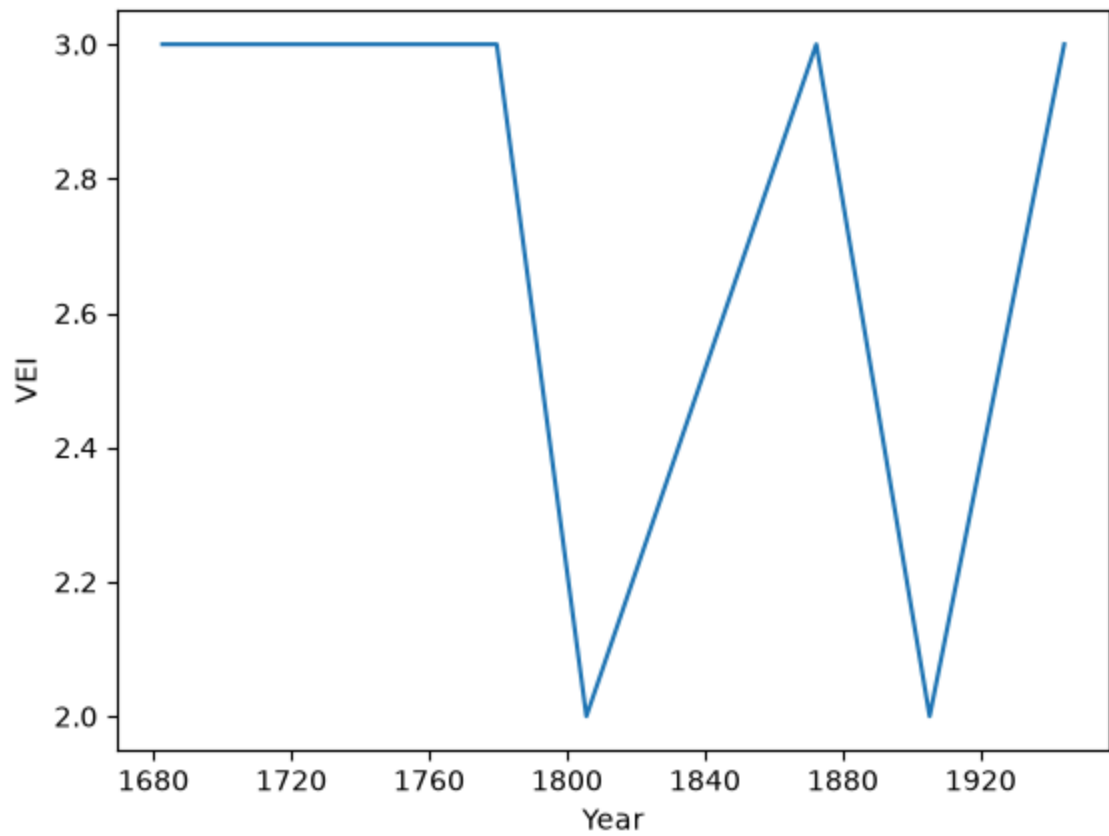




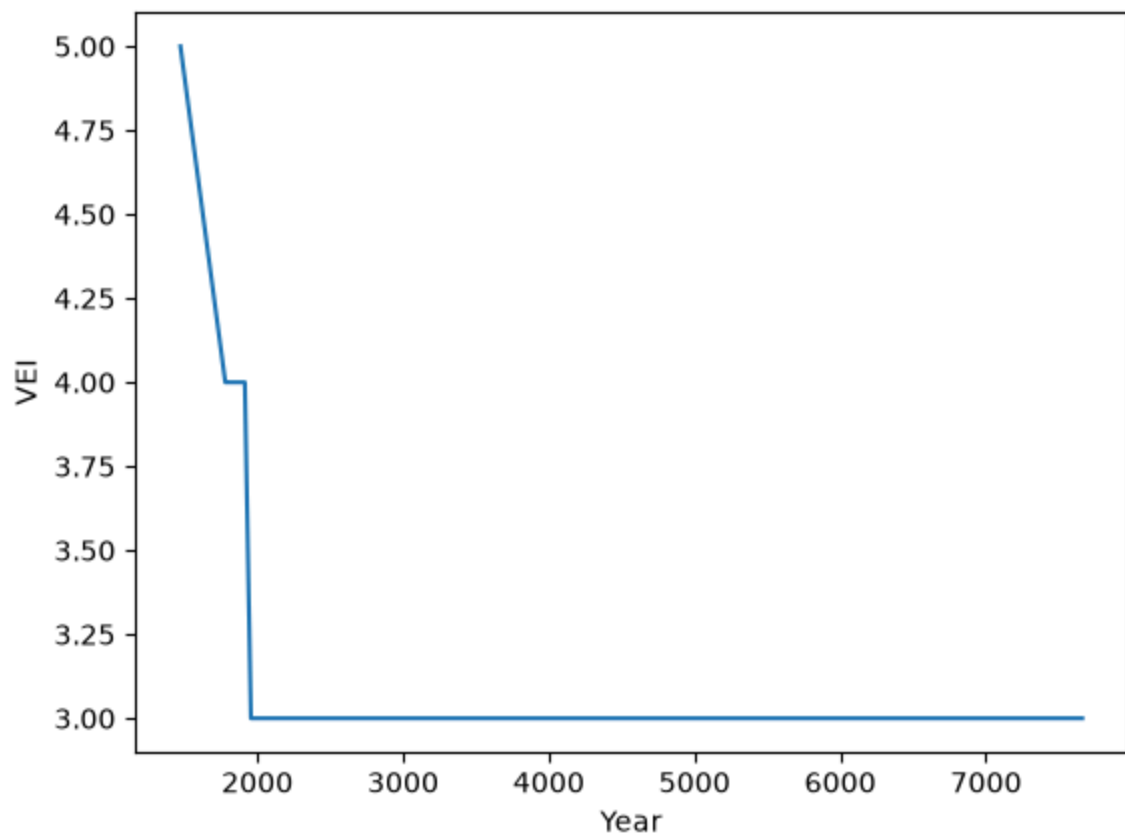
Gamalama



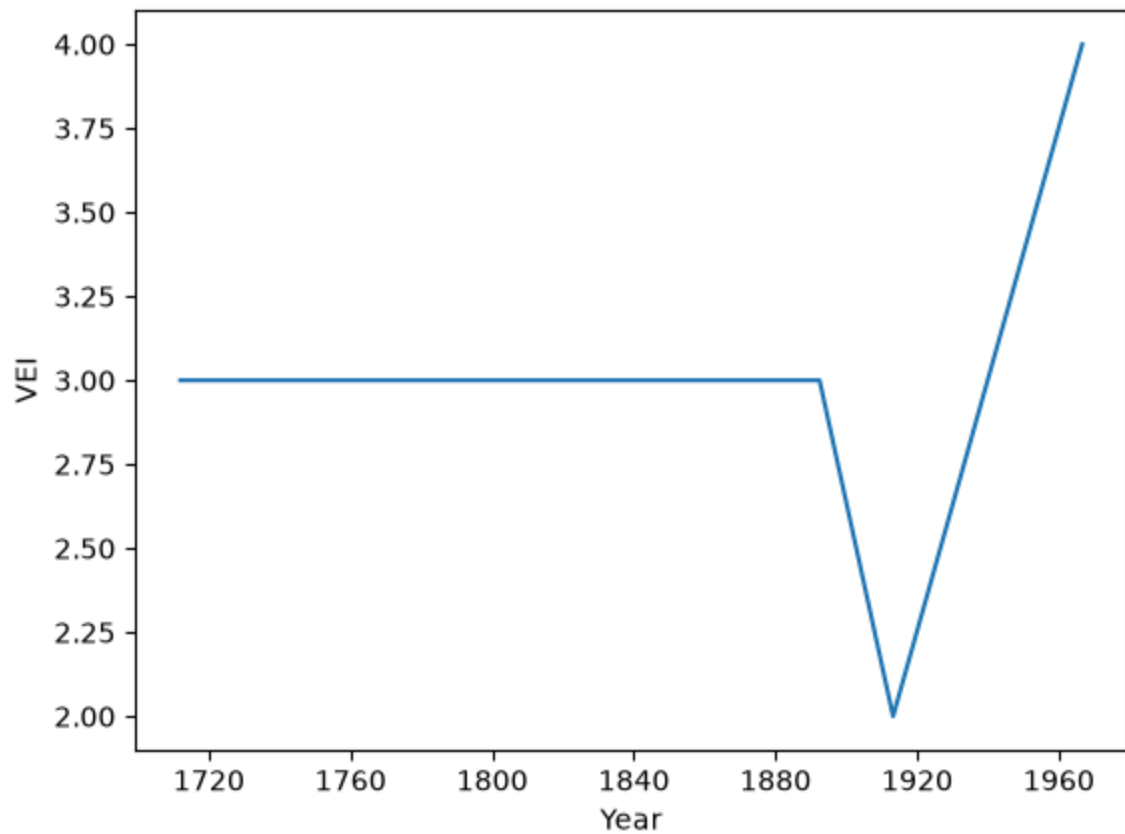
Vesuvius

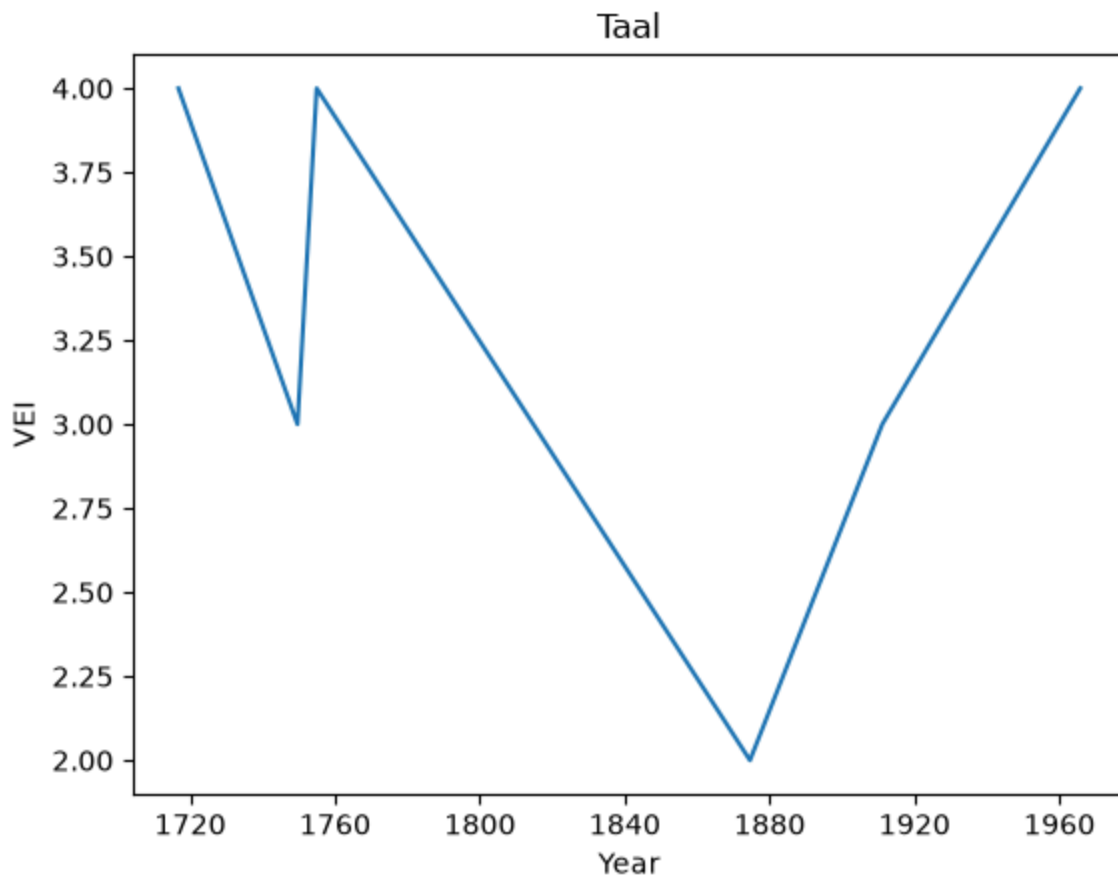


Aira



Awu





Part 3: City Jobs (15 POINTS TOTAL)

Proficiency in SQL is also super important. SQL databases are essentially relational databases in which there are vast amounts of tabular data. which can often be used to connect with related tabular data. [This](#) is a pretty good intro into learning more about SQL.

Check out this [tutorial](#) for some clarifications on SQL.



IMPORTANT: Downloading the Dataset

Please follow these steps to download the database file:

- **Step 1:** Go to the Kaggle Dataset
 - Click this link: [SF Salaries Dataset on Kaggle](#)

- **Step 2:** Create/Login to Kaggle Account
 - You need a free Kaggle account to download datasets
 - If you don't have one, sign up at kaggle.com
 - Log in to your account
- **Step 3:** Download and Extract
 - On the dataset page, click the "Download" button (upper right corner)
 - A ZIP file will be downloaded. Locate it in your system and extract it.
 - You will see a `database.sqlite` file - this is what we need.
 - Move the `database.sqlite` file to the same folder where your notebook is saved.

Now! We'll be using `sqlite` to access a database.

- Check out the description of the data so you know the table / column names.

The following code will use `sqlite3` to create a database connection. `sqlite3` is the library in Python that assists in navigating through SQL databases.

Note: If you are working on this assignment via Google Colab, sometimes the runtime resets and it will throw errors.

Instead of running through the entire notebook, run the notebook from the following code block and onwards:

- Click anywhere on the next code block.
- Go up to where it says **'Runtime'** in the toolbar (right under the title of the notebook and **in between 'Insert' and 'Tools'**)
- Hover over it and **click on the option** that says **'Run cell and below'**

```
In [17]: import sqlite3
import pandas as pd # Pandas was already imported from the previous sections

conn = sqlite3.connect("database.sqlite")
crsr = conn.cursor()
```

If you cannot see anything when executing this code, that means you did not download the Kaggle dataset correctly.

```
In [19]: # This code will let you check out the different tables within the database.
query = "SELECT name FROM sqlite_master WHERE type='table';"
tables = crsr.execute(query).fetchall()
print(tables)
```

```
[('Salaries',)]
```

Remember that each problem should be solved with a single SQL query.

Note: All outputs must be shown

- Only include whatever fields are mentioned throughout each question, nothing more and nothing less.
- Follow each instruction clearly

3.1.1: 2 Points

From the **Salaries** table, get the **average total pay** for all employees whose job titles contain the word "**clerk**" (**not case-sensitive**) between the years **2011 to 2014**.

Remember that any employee with "clerk" in their job title should be included, regardless of other words in the title.

Hint: Look into [this](#)

```
In [25]: query = '''
          SELECT AVG(TotalPay) AS TotalPay FROM Salaries
          WHERE JobTitle LIKE '%clerk%' AND Year BETWEEN 2011 AND 2014
          '''

          # KEEP THIS. It will display the whole dataframe.
          df = pd.read_sql(query, conn)
          df
```

```
Out[25]:
```

	TotalPay
0	48086.855056

3.1.2: 2 Points

From the **Salaries** table, get all the clerks (all job titles consisting of the word "clerk" (**not case-sensitive**)) in the **year 2013** making under **\$50,000 as a Total pay**. **Sort** them in **descending** order by their pay.

Remember that clerks that also occupy other professions are still considered clerks.

```
In [22]: query = '''
          SELECT JobTitle, TotalPay
          FROM Salaries
          WHERE JobTitle LIKE '%clerk%' AND YEAR == 2013 AND TotalPay < 50000
          ORDER BY TotalPay DESC
          '''

          # KEEP THIS. It will display the whole dataframe.
          df = pd.read_sql(query, conn)
          df
```

Out [22] :

	JobTitle	TotalPay
0	Senior Clerk	49949.24
1	Clerk	49923.11
2	Assessment Clerk	49917.59
3	Clerk Typist	49857.09
4	Senior Account Clerk	49831.41
5	Senior Clerk	49746.15
6	Senior Clerk	49731.28
7	Account Clerk	49677.41
8	Senior Clerk Typist	49500.05
9	Clerk	49449.05
10	Account Clerk	49446.02
11	Account Clerk	49380.13
12	Medical Records Clerk	49319.98
13	Deputy Court Clerk III	49300.50
14	Senior Clerk	49295.23
15	Senior Clerk	49291.79
16	Senior Clerk Typist	49289.84
17	Account Clerk	49275.03
18	Senior Clerk	49261.28
19	Senior Clerk	49205.71
20	Account Clerk	49198.51
21	Senior Clerk	49153.54
22	Health Care Billing Clerk 2	49083.15
23	Senior Clerk	49075.22
24	Junior Clerk	49005.00
25	Junior Clerk	49004.70
26	Senior Clerk	49000.96
27	Senior Clerk	48949.01
28	Senior Clerk	48944.61
29	Senior Clerk	48889.88
30	Clerk	48875.54
31	Clerk	48872.27

	JobTitle	TotalPay
32	Clerk	48829.19
33	Clerk	48805.93
34	Senior Clerk	48779.42
35	Principal Clerk	48764.00
36	Unit Clerk	48713.11
37	Senior Clerk	48645.11
38	Senior Clerk	48579.28
39	Account Clerk	48570.77
40	Senior Clerk	48440.00
41	Clerk	48341.55
42	Clerk	48341.53
43	Senior Clerk Typist	48306.17
44	Senior Clerk	48257.41
45	Senior Clerk	48240.03
46	Legal Process Clerk	48223.96
47	Clerk	48218.86
48	Senior Clerk Typist	48118.96
49	Senior Clerk	47974.04
50	Payroll Clerk	47961.12
51	Deputy Court Clerk II	47944.67
52	Senior Clerk	47857.16
53	Account Clerk	47840.66
54	Senior Clerk Typist	47837.15
55	Senior Clerk	47763.60
56	Senior Clerk Typist	47760.35
57	Principal Account Clerk	47759.55
58	Account Clerk	47693.87
59	Senior Clerk Typist	47562.53
60	Senior Clerk Typist	47553.82
61	Senior Clerk	47526.62
62	Clerk	47496.67
63	Clerk	47473.53

	JobTitle	TotalPay
64	Clerk	47469.11
65	Senior Clerk	47457.81
66	Senior Clerk	47343.27
67	Senior Clerk	47338.94
68	Clerk Typist	47304.60
69	Clerk	47209.58
70	Clerk	47201.71
71	Senior Clerk	47196.05
72	Deputy Court Clerk II	47181.09
73	Senior Clerk	47140.99
74	Clerk	47121.02
75	Senior Clerk	47023.80
76	Senior Water Services Clerk	47010.64
77	Clerk	46990.92
78	Clerk Typist	46956.73
79	Personnel Clerk	46951.83
80	Clerk	46933.02
81	Clerk	46908.23
82	Clerk	46908.17
83	Clerk	46899.83
84	Clerk	46850.01
85	Health Care Billing Clerk 2	46817.67
86	Account Clerk	46736.38
87	Clerk	46733.81
88	Senior Clerk Typist	46723.51
89	Senior Clerk	46624.58
90	Senior Clerk	46574.32
91	Clerk	46555.02
92	Personnel Clerk	46554.04
93	Clerk	46543.45
94	Clerk Typist	46530.72
95	Senior Clerk	46529.03

	JobTitle	TotalPay
96	Clerk	46509.72
97	Clerk Typist	46408.23
98	Clerk	46343.71
99	Senior Clerk	46269.17
100	Health Care Billing Clerk 2	46264.15
101	Principal Clerk	46203.19
102	Account Clerk	46172.97
103	Deputy Court Clerk III	46155.92
104	Clerk	46124.86
105	Senior Clerk	46034.42
106	Senior Clerk	45957.88
107	Senior Clerk	45897.77
108	Clerk	45886.10
109	Clerk	45820.83
110	Clerk	45660.07
111	Senior Account Clerk	45612.06
112	Clerk	45606.04
113	Senior Clerk	45605.64
114	Senior Clerk	45547.90
115	Deputy Court Clerk I	45543.66
116	Clerk	45526.01
117	Chief Clerk	45500.40
118	Senior Clerk	45414.09
119	Clerk	45406.23
120	Clerk	45354.57
121	Clerk	45347.90
122	Clerk	45185.43
123	Clerk	45152.90
124	Clerk	45142.36
125	Senior Account Clerk	45127.06
126	Senior Personnel Clerk	45110.12
127	Senior Clerk	44986.84

	JobTitle	TotalPay
128	Health Care Billing Clerk 2	44984.50
129	Clerk	44864.87
130	Account Clerk	44856.63
131	Senior Clerk	44841.78
132	Medical Records Clerk	44688.06
133	Principal Clerk	44639.70
134	Clerk Typist	44553.80
135	Senior Clerk Typist	44496.73
136	Assessment Clerk	44386.50
137	Senior Legal Process Clerk	44364.28
138	Deputy Court Clerk I	44288.28
139	Senior Clerk	44259.51
140	Clerk	44250.28
141	Clerk	44242.95
142	Senior Clerk	44100.47
143	Unit Clerk	44076.94
144	Clerk	44023.35
145	Clerk	44005.23
146	Senior Account Clerk	43910.13
147	Senior Clerk Typist	43875.00
148	Clerk	43776.12
149	Clerk	43776.11
150	Clerk	43776.10
151	Account Clerk	43768.05
152	Clerk	43595.76
153	Clerk	43511.34
154	Account Clerk	43506.74
155	Clerk	43460.11
156	Account Clerk	43450.56
157	Clerk	43444.67
158	Clerk	43442.88
159	Junior Clerk	43366.97

	JobTitle	TotalPay
160	Sr Payroll & Personnel Clerk	43344.01
161	Senior Clerk Typist	43213.23
162	Senior Clerk Typist	43180.83
163	Clerk Typist	42987.62
164	Senior Clerk	42949.50
165	Deputy Court Clerk I	42938.25
166	Clerk	42879.05
167	Senior Clerk	42788.27
168	Principal Clerk	42760.26
169	Senior Clerk	42681.02
170	Clerk	42611.71
171	Clerk Typist	42588.00
172	Account Clerk	42485.97
173	Clerk	42442.75
174	Principal Clerk	42413.23
175	Elections Clerk	42187.92
176	Senior Clerk	41917.61
177	Clerk	41912.57
178	Deputy Court Clerk II	41875.07
179	Clerk Typist	41741.51
180	Senior Clerk	41466.64
181	Health Care Billing Clerk 2	41321.97
182	Clerk	41225.32
183	Account Clerk	41160.22
184	Legal Process Clerk	41039.24
185	Senior Clerk	40964.02
186	Elections Clerk	40938.32
187	Senior Clerk	40912.38
188	Clerk	40807.41
189	Senior Clerk	40676.70
190	Clerk	40620.58
191	Senior Clerk	40466.36

	JobTitle	TotalPay
192	Senior Clerk Typist	40416.96
193	Health Care Billing Clerk 2	40383.28
194	Senior Clerk Typist	40182.23
195	Senior Clerk Typist	39923.48
196	Senior Clerk	39877.22
197	Sr Transit Information Clerk	39871.80
198	Senior Clerk	39822.30
199	Account Clerk	39810.24
200	Clerk Typist	39661.47
201	Unit Clerk	39654.47
202	Clerk	39573.95
203	Clerk	39465.04
204	Unit Clerk	39166.77
205	Elections Clerk	39022.59
206	Clerk	39020.00
207	Elections Clerk	39008.82
208	Personnel Clerk	38956.60
209	Sr Transit Information Clerk	38897.52
210	Senior Clerk	38851.80
211	Junior Clerk	38834.61
212	Clerk	38734.86
213	Principal Clerk	38703.38
214	Senior Clerk	38318.03
215	Deputy Court Clerk I	38168.46
216	Deputy Court Clerk I	38130.60
217	Clerk	38014.21
218	Senior Account Clerk	37961.42
219	Senior Clerk	37792.37
220	Deputy Court Clerk I	37727.50
221	Deputy Court Clerk I	37694.41
222	Clerk	37654.31
223	Health Care Billing Clerk 1	37620.01

	JobTitle	TotalPay
224	Senior Clerk	37366.18
225	Payroll Clerk	37165.85
226	Health Care Billing Clerk 2	37155.45
227	Deputy Court Clerk I	37137.98
228	Senior Clerk	37082.11
229	Clerk	36689.14
230	Elections Clerk	36653.83
231	Clerk	36588.63
232	Senior Clerk	36485.44
233	Senior Clerk	36485.42
234	Senior Clerk	36485.40
235	Senior Account Clerk	36409.07
236	Senior Clerk	36404.38
237	Senior Personnel Clerk	36305.82
238	Senior Clerk Typist	36231.52
239	Clerk Typist	36168.90
240	Senior Clerk	36131.42
241	Senior Clerk	36126.20
242	Payroll Clerk	36051.75
243	Senior Clerk	36048.38
244	Senior Clerk Typist	35919.04
245	Senior Account Clerk	35892.01
246	Deputy Court Clerk II	35858.92
247	Senior Clerk	35751.87
248	Junior Clerk	35751.01
249	Deputy Court Clerk I	35724.49
250	Senior Clerk Typist	35616.01
251	Sr Payroll & Personnel Clerk	35589.56
252	Account Clerk	35576.19
253	Senior Clerk	35485.81
254	Senior Account Clerk	35400.11
255	Senior Clerk	35365.05

	JobTitle	TotalPay
256	Clerk	35262.33
257	Clerk	35193.30
258	Clerk	35193.30
259	Senior Account Clerk	35184.82
260	Senior Water Services Clerk	35139.10
261	Senior Clerk	35126.58
262	Senior Clerk Typist	34752.13
263	Clerk Typist	34676.68
264	Clerk	34620.82
265	Junior Clerk	34596.09
266	Deputy Court Clerk I	34439.68
267	Senior Clerk	34391.00
268	Junior Clerk	34389.53
269	Clerk	34370.05
270	Sr Payroll & Personnel Clerk	34212.53
271	Senior Clerk	34153.02
272	Junior Clerk	34090.83
273	Health Care Billing Clerk 2	34057.75
274	Payroll Clerk	34021.50
275	Deputy Court Clerk I	34021.00
276	Junior Clerk	33962.94
277	Clerk Typist	33887.90
278	Senior Clerk	33686.78
279	Junior Clerk	33568.49
280	Clerk	33488.75
281	Senior Clerk	33320.02
282	Senior Clerk	33320.01
283	Junior Clerk	33271.68
284	Clerk	33180.02
285	Account Clerk	33174.96
286	Senior Clerk	32974.92
287	Junior Clerk	32886.01

	JobTitle	TotalPay
288	Junior Clerk	32761.98
289	Senior Clerk	32754.03
290	Account Clerk	32722.06
291	Senior Clerk Typist	32614.55
292	Clerk Typist	32587.21
293	Account Clerk	32567.32
294	Clerk	32401.14
295	Clerk	32140.00
296	Senior Clerk	31955.84
297	Principal Clerk	31954.23
298	Junior Clerk	31512.92
299	Senior Clerk	31487.41
300	Personnel Clerk	31457.49
301	Personnel Clerk	31451.20
302	Medical Records Clerk	31414.57
303	Assessment Clerk	31387.05
304	Medical Records Clerk	31337.81
305	Legal Process Clerk	31196.10
306	Permit and Citation Clerk	31157.20
307	Clerk	31150.67
308	Unit Clerk	30925.99
309	Unit Clerk	30577.99
310	Clerk	30533.01
311	Senior Legal Process Clerk	30247.78
312	Senior Clerk Typist	30227.37
313	Senior Account Clerk	30155.92
314	Deputy Court Clerk I	30110.37
315	Clerk Typist	30054.87
316	Sr Payroll & Personnel Clerk	30011.15
317	Personnel Clerk	29956.19
318	Clerk	29860.09
319	Sr Payroll & Personnel Clerk	29790.55

	JobTitle	TotalPay
320	Senior Clerk	29630.33
321	Clerk	29621.07
322	Personnel Clerk	29498.31
323	Clerk	29242.38
324	Clerk	29197.18
325	Deputy Court Clerk I	28976.50
326	Clerk	28926.01
327	Clerk	28926.01
328	Clerk	28926.00
329	Senior Clerk	28694.29
330	Clerk	28674.91
331	Senior Clerk	28571.91
332	Clerk	28495.60
333	Junior Clerk	28392.03
334	Senior Account Clerk	28136.11
335	Account Clerk	27974.01
336	Junior Clerk	27762.27
337	Senior Clerk Typist	27728.65
338	Personnel Clerk	27725.57
339	Legal Process Clerk	27358.96
340	Clerk	27268.51
341	Clerk Typist	27163.38
342	Junior Clerk	26986.84
343	Senior Clerk	26953.74
344	Junior Clerk	26910.18
345	Account Clerk	26606.90
346	Principal Clerk	26420.80
347	Senior Legal Process Clerk	26390.28
348	Clerk	26179.03
349	Medical Records Clerk	26109.88
350	Senior Clerk	26018.08
351	Account Clerk	25672.72

	JobTitle	TotalPay
352	Senior Clerk Typist	25452.95
353	Medical Records Clerk	25440.40
354	Deputy Court Clerk II	25440.37
355	Senior Clerk Typist	25368.49
356	Personnel Clerk	25049.05
357	Account Clerk	24991.58
358	Medical Records Clerk	24883.72
359	Senior Account Clerk	24866.87
360	Deputy Court Clerk II	24776.52
361	Medical Records Clerk	24516.10
362	Senior Account Clerk	24414.06
363	Senior Clerk	24280.01
364	Legal Process Clerk	24159.85
365	Unit Clerk	23981.36
366	Senior Account Clerk	23928.00
367	Payroll Clerk	23878.96
368	Senior Account Clerk	23798.05
369	Senior Clerk Typist	23571.66
370	Deputy Court Clerk I	23562.05
371	Deputy Court Clerk I	23539.33
372	Senior Account Clerk	23531.45
373	Senior Clerk Typist	23505.47
374	Deputy Court Clerk I	23442.75
375	Medical Records Clerk	23403.25
376	Senior Clerk Typist	23322.38
377	Payroll Clerk	23197.20
378	Medical Records Clerk	23146.37
379	Clerk Typist	23010.19
380	Senior Clerk Typist	22929.18
381	Clerk	22828.92
382	Clerk	22826.84
383	Senior Clerk Typist	22704.40

	JobTitle	TotalPay
384	Payroll Clerk	22664.46
385	Senior Clerk Typist	22326.76
386	Deputy Court Clerk I	22233.24
387	Clerk	22142.40
388	Senior Clerk	22086.04
389	Senior Clerk	22079.25
390	Medical Records Clerk	21973.22
391	Senior Clerk	21658.00
392	Senior Clerk Typist	21617.26
393	Senior Clerk Typist	21560.04
394	Senior Clerk Typist	21502.82
395	Unit Clerk	21472.72
396	Senior Water Services Clerk	21419.74
397	Clerk	21262.62
398	Clerk	21257.60
399	Principal Clerk	21175.60
400	Senior Account Clerk	21094.86
401	Senior Account Clerk	21074.25
402	Senior Account Clerk	21061.67
403	Senior Clerk Typist	21056.50
404	Clerk	21053.91
405	Senior Clerk Typist	20896.30
406	Medical Records Clerk	20883.38
407	Deputy Court Clerk II	20835.69
408	Clerk	20770.49
409	Deputy Court Clerk I	20679.12
410	Senior Legal Process Clerk	20586.62
411	Clerk	20554.53
412	Deputy Court Clerk I	20543.40
413	Senior Clerk Typist	20484.32
414	Deputy Court Clerk I	20401.38
415	Deputy Court Clerk II	20364.28

	JobTitle	TotalPay
416	Legal Process Clerk	20309.23
417	Clerk Typist	20186.15
418	Deputy Court Clerk II	20152.82
419	Account Clerk	20151.15
420	Clerk	20087.50
421	Medical Records Clerk	20084.04
422	Senior Clerk	19992.00
423	Senior Clerk Typist	19881.71
424	Deputy Court Clerk I	19770.75
425	Senior Clerk	19761.17
426	Clerk Typist	19724.34
427	Deputy Court Clerk I	19638.22
428	Senior Clerk Typist	19614.62
429	Deputy Court Clerk I	19583.27
430	Deputy Court Clerk I	19537.82
431	Senior Clerk Typist	19524.98
432	Medical Records Clerk	19518.54
433	Clerk Typist	19476.39
434	Senior Clerk Typist	19380.04
435	Senior Clerk	19358.48
436	Deputy Court Clerk I	19333.30
437	Senior Legal Process Clerk	19272.60
438	Unit Clerk	19207.09
439	Clerk	19173.52
440	Senior Clerk Typist	19168.28
441	Senior Clerk Typist	18956.34
442	Deputy Court Clerk I	18907.20
443	Deputy Court Clerk I	18725.40
444	Health Care Billing Clerk 2	18282.98
445	Senior Clerk Typist	18185.62
446	Clerk	18169.15
447	Personnel Clerk	17870.01

	JobTitle	TotalPay
448	Chief Clerk	17858.02
449	Clerk Typist	17793.54
450	Clerk	17526.80
451	Elections Clerk	17521.95
452	Senior Clerk	17480.00
453	Senior Clerk	17477.31
454	Clerk	17456.05
455	Medical Records Clerk	17449.16
456	Clerk	17089.44
457	Clerk	17067.66
458	Clerk	16904.00
459	Clerk	16873.50
460	Payroll Clerk	16700.92
461	Senior Clerk	16618.35
462	Deputy Court Clerk I	16282.50
463	Elections Clerk	16179.80
464	Clerk	16160.41
465	Senior Clerk	15981.70
466	Junior Clerk	15803.90
467	Clerk Typist	15661.52
468	Chief Clerk	15552.10
469	Clerk	15371.69
470	Senior Clerk	15254.33
471	Senior Clerk	15128.06
472	Senior Clerk	15103.33
473	Senior Clerk	15072.75
474	Senior Clerk	15045.90
475	Clerk	14945.10
476	Clerk	14924.76
477	Senior Clerk	14702.46
478	Senior Clerk	14692.05
479	Clerk	14689.15

	JobTitle	TotalPay
480	Senior Water Services Clerk	14686.83
481	Account Clerk	14432.40
482	Clerk	14223.10
483	Junior Clerk	14082.10
484	Deputy Court Clerk I	13879.50
485	Account Clerk	13836.80
486	Senior Clerk	13356.00
487	Clerk	13310.56
488	Senior Clerk	13244.70
489	Senior Clerk	13187.46
490	Senior Clerk	13109.34
491	Transit Information Clerk	13061.43
492	Clerk	13026.77
493	Clerk	13001.65
494	Clerk	12908.44
495	Elections Clerk	12791.87
496	Senior Account Clerk	12703.85
497	Senior Clerk	12657.35
498	Elections Clerk	12654.75
499	Senior Clerk Typist	12584.53
500	Junior Clerk	12447.38
501	Senior Clerk	12255.52
502	Senior Legal Process Clerk	12060.03
503	Clerk	12039.10
504	Clerk Typist	11853.42
505	Senior Clerk	11662.00
506	Account Clerk	11545.09
507	Senior Clerk	11542.79
508	Elections Clerk	11537.38
509	Junior Clerk	11077.50
510	Elections Clerk	10738.96
511	Elections Clerk	10539.80

	JobTitle	TotalPay
512	Personnel Clerk	10318.00
513	Clerk	10279.78
514	Senior Clerk	10227.30
515	Payroll Clerk	10162.35
516	Clerk	10139.18
517	Clerk Typist	10134.82
518	Senior Clerk	10115.00
519	Clerk	10063.84
520	Clerk	10041.24
521	Clerk	10003.58
522	Clerk	9958.38
523	Clerk	9862.79
524	Personnel Clerk	9778.24
525	Junior Clerk	9743.20
526	Clerk	9742.44
527	Junior Clerk	9632.80
528	Clerk	9601.83
529	Unit Clerk	9500.00
530	Senior Clerk	9402.51
531	Clerk	9343.43
532	Senior Clerk	9329.60
533	Account Clerk	9304.20
534	Clerk Typist	9174.59
535	Junior Clerk	9160.45
536	Unit Clerk	9152.27
537	Junior Clerk	9129.69
538	Senior Clerk Typist	9127.95
539	Clerk	9107.17
540	Junior Clerk	9088.16
541	Clerk	8946.34
542	Elections Clerk	8812.21
543	Senior Legal Process Clerk	8804.41

	JobTitle	TotalPay
544	Elections Clerk	8699.06
545	Clerk	8672.78
546	Clerk	8597.46
547	Account Clerk	8442.71
548	Senior Legal Process Clerk	8349.83
549	Senior Clerk Typist	8236.20
550	Clerk	8215.79
551	Senior Clerk	8163.40
552	Clerk	8060.11
553	Clerk	8029.97
554	Clerk	7909.45
555	Elections Clerk	7835.57
556	Principal Clerk	7782.51
557	Clerk	7670.92
558	Clerk	7578.01
559	Senior Clerk	7547.97
560	Legal Process Clerk	7412.00
561	Clerk	7291.76
562	Senior Personnel Clerk	7255.03
563	Elections Clerk	7126.69
564	Unit Clerk	7014.99
565	Clerk	6927.68
566	Clerk	6920.51
567	Deputy Court Clerk II	6765.13
568	Account Clerk	6681.15
569	Elections Clerk	6659.17
570	Senior Clerk Typist	6646.51
571	Medical Records Clerk	6555.05
572	Clerk	6347.65
573	Senior Clerk	6257.91
574	Clerk	6217.08
575	Clerk	6046.34

	JobTitle	TotalPay
576	Junior Clerk	5991.09
577	Senior Legal Process Clerk	5889.66
578	Junior Clerk	5889.53
579	Clerk	5855.02
580	Account Clerk	5841.36
581	Deputy Court Clerk III	5706.37
582	Junior Clerk	5663.38
583	Senior Clerk Typist	5653.20
584	Deputy Court Clerk II	5650.25
585	Junior Clerk	5649.65
586	Clerk	5606.92
587	Clerk	5596.88
588	Clerk	5550.63
589	Patient Accounts Clerk	5518.22
590	Clerk	5443.19
591	Payroll Clerk	5401.96
592	Senior Clerk	5331.20
593	Junior Clerk	5305.67
594	Clerk	5235.31
595	Junior Clerk	5124.69
596	Junior Clerk	5098.04
597	Legal Process Clerk	5057.60
598	Principal Clerk	5050.30
599	Clerk Typist	5030.71
600	Junior Clerk	5003.34
601	Assessment Clerk	4951.69
602	Clerk	4942.93
603	Junior Clerk	4938.71
604	Elections Clerk	4856.17
605	Junior Clerk	4844.10
606	Senior Clerk	4831.41
607	Junior Clerk	4772.55

	JobTitle	TotalPay
608	Senior Clerk	4748.11
609	Junior Clerk	4701.01
610	Senior Clerk	4699.78
611	Senior Legal Process Clerk	4652.00
612	Senior Clerk	4635.67
613	Clerk Typist	4573.21
614	Account Clerk	4485.18
615	Junior Clerk	4430.99
616	Elections Clerk	4428.34
617	Personnel Clerk	4210.96
618	Junior Clerk	4205.44
619	Senior Clerk	4185.83
620	Unit Clerk	4053.31
621	Principal Clerk	4044.00
622	Senior Clerk Typist	3982.28
623	Junior Clerk	3935.54
624	Senior Personnel Clerk	3920.56
625	Junior Clerk	3858.67
626	Deputy Court Clerk II	3840.87
627	Senior Clerk	3831.80
628	Clerk	3696.10
629	Junior Clerk	3641.73
630	Account Clerk	3542.92
631	Clerk	3363.25
632	Clerk	3294.35
633	Clerk	3267.92
634	Senior Clerk	3238.29
635	Junior Clerk	3138.63
636	Senior Clerk	2998.80
637	Senior Clerk	2998.80
638	Senior Clerk	2998.80
639	Senior Clerk	2998.80

	JobTitle	TotalPay
640	Elections Clerk	2984.29
641	Clerk	2942.82
642	Senior Clerk Typist	2929.60
643	Senior Clerk Typist	2929.60
644	Health Care Billing Clerk 2	2875.50
645	Junior Clerk	2787.84
646	Junior Clerk	2787.84
647	Clerk	2751.46
648	Junior Clerk	2621.68
649	Elections Clerk	2619.77
650	Clerk	2601.33
651	Account Clerk	2509.12
652	Junior Clerk	2457.82
653	Senior Clerk	2385.27
654	Senior Clerk Typist	2371.00
655	Junior Clerk	2365.51
656	Junior Clerk	2363.20
657	Clerk	2352.75
658	Junior Clerk	2335.50
659	Clerk	2326.57
660	Junior Clerk	2310.12
661	Junior Clerk	2169.35
662	Deputy Court Clerk II	2129.21
663	Junior Clerk	2088.57
664	Senior Clerk	2084.71
665	Junior Clerk	2081.65
666	Junior Clerk	2079.34
667	Junior Clerk	2077.04
668	Junior Clerk	2070.11
669	Junior Clerk	2070.11
670	Junior Clerk	2063.19
671	Junior Clerk	2047.03

	JobTitle	TotalPay
672	Unit Clerk	2007.66
673	Junior Clerk	2003.18
674	Personnel Clerk	1970.00
675	Account Clerk	1871.50
676	Account Clerk	1723.00
677	Senior Clerk	1674.03
678	Senior Account Clerk	1595.20
679	Clerk	1411.15
680	Junior Clerk	1236.99
681	Clerk	1124.90
682	Junior Clerk	1015.44
683	Clerk	964.20
684	Junior Clerk	948.51
685	Junior Clerk	939.28
686	Junior Clerk	936.97
687	Junior Clerk	920.82
688	Unit Clerk	887.86
689	Junior Clerk	851.59
690	Junior Clerk	833.12
691	Senior Clerk	833.00
692	Junior Clerk	807.73
693	Junior Clerk	803.12
694	Junior Clerk	791.58
695	Junior Clerk	784.66
696	Junior Clerk	775.43
697	Junior Clerk	722.35
698	Junior Clerk	680.80
699	Junior Clerk	678.50
700	Junior Clerk	676.19
701	Junior Clerk	669.26
702	Senior Clerk	666.40
703	Junior Clerk	664.65

	JobTitle	TotalPay
704	Junior Clerk	664.65
705	Junior Clerk	664.65
706	Junior Clerk	662.34
707	Junior Clerk	657.73
708	Junior Clerk	657.73
709	Junior Clerk	657.73
710	Junior Clerk	657.73
711	Junior Clerk	650.80
712	Junior Clerk	650.80
713	Junior Clerk	650.80
714	Junior Clerk	650.80
715	Junior Clerk	650.80
716	Junior Clerk	643.88
717	Junior Clerk	643.88
718	Junior Clerk	643.88
719	Junior Clerk	643.88
720	Junior Clerk	643.88
721	Junior Clerk	636.96
722	Junior Clerk	636.96
723	Junior Clerk	636.96
724	Junior Clerk	636.96
725	Junior Clerk	636.96
726	Junior Clerk	636.96
727	Junior Clerk	636.96
728	Junior Clerk	630.03
729	Junior Clerk	630.03
730	Junior Clerk	623.11
731	Junior Clerk	623.11
732	Junior Clerk	613.88
733	Junior Clerk	613.87
734	Junior Clerk	609.26
735	Junior Clerk	593.10

	JobTitle	TotalPay
736	Junior Clerk	590.80
737	Junior Clerk	466.18
738	Junior Clerk	466.18
739	Sr Payroll & Personnel Clerk	450.40
740	Clerk Typist	391.15
741	Junior Clerk	369.25
742	Junior Clerk	369.25
743	Junior Clerk	369.25
744	Junior Clerk	369.25
745	Junior Clerk	369.25
746	Junior Clerk	369.25
747	Junior Clerk	369.25
748	Junior Clerk	350.79
749	Senior Clerk	333.20
750	Junior Clerk	327.71
751	Junior Clerk	309.24
752	Unit Clerk	246.85
753	Principal Account Clerk	172.56
754	Deputy Court Clerk III	164.75
755	Senior Clerk Typist	35.62
756	Senior Assessment Clerk	20.73
757	Senior Clerk	5.56
758	Account Clerk	0.00
759	Senior Clerk Typist	0.00
760	Clerk	0.00
761	Senior Clerk Typist	0.00
762	Junior Clerk	0.00
763	Clerk Typist	0.00
764	Personnel Clerk	0.00
765	Clerk Typist	0.00
766	Inventory Clerk	0.00
767	Principal Clerk	0.00

	JobTitle	TotalPay
768	Senior Clerk Typist	0.00
769	Clerk Typist	0.00
770	Clerk	0.00
771	Senior Legal Process Clerk	0.00
772	Senior Account Clerk	0.00
773	Senior Clerk Typist	0.00
774	Senior Clerk Typist	0.00
775	Elections Clerk	0.00
776	Junior Clerk	0.00
777	Deputy Court Clerk II	0.00
778	Senior Personnel Clerk	0.00
779	Chf Payroll & Personnel Clerk	0.00
780	Senior Clerk	0.00
781	Deputy Court Clerk II	0.00
782	Clerk Typist	0.00
783	Clerk Typist	0.00
784	Chf Payroll & Personnel Clerk	0.00
785	Senior Clerk Typist	0.00
786	Clerk Typist	0.00

3.1.3: 4 Points

From the **Salaries** table, first get the **averages** of **base pay**, **benefits**, and **overtime pay** for clerks (all job titles consisting of the word "clerk" **(not case-sensitive)**).

- Then, make a **column with the sum** of these **three averages**
- Finally, **exclude** job titles containing "SENIOR" **(case-sensitive)**

Remember that clerks that also occupy other professions are still considered clerks.

Example:

Let's say the Salaries table has these records:

JobTitle	BasePay	Benefits	OvertimePay
CLERK	40000	15000	5000

JobTitle	BasePay	Benefits	OvertimePay
Clerk	35000	12000	3000
ACCOUNTING CLERK	45000	18000	6000
SENIOR CLERK	60000	25000	8000
Senior Clerk	55000	22000	7000
MANAGER	80000	30000	10000

Which rows to include:

- Rows 1, 2, 3, 5 (all contain "clerk" in any case)
- Exclude Row 4 (contains uppercase "SENIOR")
- Exclude Row 6 (doesn't contain "clerk")

Calculate averages from rows 1, 2, 3, 5:

- $\text{avg_base_pay} = (40000 + 35000 + 45000 + 55000) / 4 = 43750$
- $\text{avg_benefits} = (15000 + 12000 + 18000 + 22000) / 4 = 16750$
- $\text{avg_overtime_pay} = (5000 + 3000 + 6000 + 7000) / 4 = 5250$
- $\text{total_avg_compensation} = 43750 + 16750 + 5250 = 65750$

Your output:

avg_base_pay	avg_benefits	avg_overtime_pay	total_avg_compensation
43750	16750	5250	65750

Note: Actual values from the database will be different!

```
In [28]: query = '''
          SELECT AVG(BasePay) AS avg_base_pay, AVG(Benefits) AS avg_benefits, AVG(
          FROM Salaries
          WHERE JobTitle LIKE '%clerk%' AND JobTitle NOT LIKE '%SENIOR%'
          '''

# KEEP THIS. It will display the whole dataframe.
df = pd.read_sql(query, conn)
df
```

```
Out[28]:
```

	avg_base_pay	avg_benefits	avg_overtime_pay
0	45188.292242	15170.247828	1075.427168

3.1.4: Creating Year-Based Tables (7 Points)

Finally, we'll make our own year based- table in our database. We already have the "Salaries" table above that contains data for multiple years (2010–2015).

Task: Create separate tables for each year (2010–2015) and add them back to the database.

- Split the Salaries table into **six year-based tables (2010–2015)**.
- Each table should contain **only the rows from that year**.
- You **must use SQL queries** to retrieve data from the Salaries table.
- You may use Python and pandas to store results and **write tables back** to the database.
- Using a Python **loop** may be helpful.
- You may use multiple lines of python code for this problem.

Hint: Check [this](#) out

Think about how you've already used SQL to read data into pandas, and how pandas can write DataFrames back into a database.

```
In [ ]: # REMOVE THIS CONTENT AND ANSWER IN YOUR OWN WAY
```

```
In [ ]: # Run this code to check if you successfully added your table.
cursor = conn.cursor()
cursor.execute("SELECT name FROM sqlite_master WHERE type='table';")
print(cursor.fetchall())
```



Part 4: Pandas 'Group By' (50 points)

This flowchart is taken from our lecture class presentation and illustrates the process of transforming data using the Pandas GroupBy operation. First, the data is input, followed by applying the GroupBy function to one or more columns of the DataFrame. Once the data is grouped, an aggregation function (such as sum(), mean(), or count()) is applied to compute summary statistics for each group



Your task is to translate the workflow shown in the flowchart into Pandas queries that perform these operations step by step.

Notes: Your task is to translate the workflow shown in the flowchart into Pandas queries. Ensure that the **exact input and exact output** from the flowchart are replicated using Pandas queries, step by step.

4.1 (Point 10) Create a sample dataset that includes columns for account, order, and ext price.

```
In [26]: # Create the Sample dataset from above flowchart
input_data = {
    'account': ['383080', '383080', '383080', '412290', '412290', '412290', '412290', '412290', '412290', '412290'],
    'order': ['10001', '10001', '10001', '10005', '10005', '10005', '10005', '10005', '10005', '10005'],
    'ext_price': [235.83, 232.32, 107.97, 2679.36, 286.02, 832.95, 3472.04, 2679.36, 286.02, 832.95]
}

# Create DataFrame
df = pd.DataFrame(input_data)

# Display Dataframe (DONT REMOVE THE CODE)
#We could also use #print(df) which is not as nice or display(df) which is t
df
```

Out[26]:		account	order	ext_price
	0	383080	10001	235.83
	1	383080	10001	232.32
	2	383080	10001	107.97
	3	412290	10005	2679.36
	4	412290	10005	286.02
	5	412290	10005	832.95
	6	412290	10005	3472.04
	7	412290	10005	915.12
	8	218895	10006	3061.12
	9	218895	10006	518.65
	10	218895	10006	216.90
	11	218895	10006	-72.18

4.2 (Point 10+10 =20) Group by order and show the intermediate results

```
In [29]: # Group by 'order' and show intermediate re
```

```
grouped_df = df.groupby('order')

# Display intermediate result for each group; hints: you have to use 'for' loop
for order_number, order_df in grouped_df:
    display(order_df)
```

	account	order	ext_price
0	383080	10001	235.83
1	383080	10001	232.32
2	383080	10001	107.97

	account	order	ext_price
3	412290	10005	2679.36
4	412290	10005	286.02
5	412290	10005	832.95
6	412290	10005	3472.04
7	412290	10005	915.12

	account	order	ext_price
8	218895	10006	3061.12
9	218895	10006	518.65
10	218895	10006	216.90
11	218895	10006	-72.18

4.3 (Point 10) Apply the Sum Aggregation for Each Group

Now we'll apply the sum aggregation to get the total ext price for each order:

```
In [36]: # Repeat group by 'order' again and then apply aggregation (sum of 'ext_price')
grouped_df = df.groupby('order')
order_sum = grouped_df['ext_price'].sum()

# Show the aggregated result after re-grouping (DONT REMOVE THE CODE)

order_sum
```

```
Out[36]: order
10001      576.12
10005     8185.49
10006     3724.49
Name: ext_price, dtype: float64
```

4.4 (Point 10) Combine the Results into One Final Table Finally, we will **reset the index** and create a combined table that shows order and the sum of the ext price for each

group:

Notes: In pandas, 'reset_index()' is a method used to reset the index of a DataFrame to its default integer-based index. By default, when you perform certain operations like groupby(), the resulting DataFrame may have a new index (e.g., the grouped column). The reset_index() method allows you to convert the current index back to a default sequential integer index and optionally, move the current index values into a regular column.

```
In [37]: # Reset index to combine result into a single DataFrame
df_reseted = order_sum.reset_index()
# Rename the columns for clarity
df_renamed = df_reseted.rename(columns = {'ext_price': 'Order_Total'})

# Show the final result (DONT REMOVE THE CODE)
df_renamed
```

```
Out [37]:
```

	order	Order_Total
0	10001	576.12
1	10005	8185.49
2	10006	3724.49

Bonus(1 point): Assignment Submission with Jupyter Notebook file(ipynb) and PDF

In addition to the instruction of the special bonus question included exclusively in Assignment 1, we would like to highlight the importance of developing good practices for assignment submission in data science projects.

Think of yourself as a reviewer of your peers' work: what would you pay attention to if you had to grade their assignments? Clearly, the raw dataset itself is not the main point—why would someone need to see every line of data you downloaded? Data science is about analysis and insight, not just data collection. What truly matters is how you analyze the data and how you present your results.

For a clean and effective submission, please avoid printing entire datasets unless they are directly relevant to the presentation of your output or if you are asked to do so. Instead, focus on:

- Showing your algorithm with clear, concise code
- Displaying the key results you want us to evaluate (or the outputs you relied on while working)
- Adding a short explanation when asked to interpret your findings

For this bonus question, please submit both your .ipynb file and the PDF version. To create a PDF: go to File (top left corner in Jupyter Notebook) → Print → Destination → Save as PDF.

Are you noticing an overwhelming amount of unnecessary printed results in your PDF? This is exactly why we emphasize the importance of cleaning up your work before submission. Removing trivial data printing makes your notebook look much more professional and easier to review—just like proofreading an essay for good grammar.

Your Homework 1 submission should be lightweight and focused, since this is a starter project designed to help you ease into the semester.

This practice helps us grade more efficiently and ensures fairness, since we can always reference your original notebook if regrading is needed.